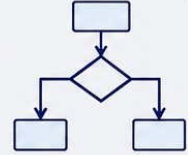


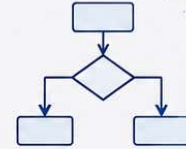
1010101010  
0101010101  
1010101010  
0101010101



# Notes



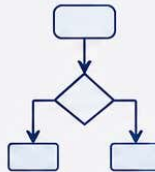
1010101010  
0101010101  
1010101010  
0101010101



# Unit - I



10110  
01001  
10101





# Introduction to Database

## 1. What is Database ?

- A database is an **organized collection** of **related data** that is stored in a computer **system** in a **systematic way** so that it can be easily accessed, managed, updated and retrieved when needed.
- A database represents a real-world entity (like students, employees, products, etc.) in the form of structured data.
- It reduces **redundancy** and maintains data **consistency**.

## 2. Data - Meaning and Definition

- Data is a collection of **raw facts** and figures about an **event**, object or **entity**.
- Data can be numbers, text, images, symbols, or any other form of facts.
- Data by itself does **not convey** any specific meaning.

### Examples of Data :

- 101
- Amit
- 72.5
- Akola
- 12-09-2026

The above values are just **raw facts**. They do not tell us what they mean until we process them.

## 3. Information - Meaning and Definition

- Information** is the **processed form** of **data** which has a **specific meaning**.
- It is obtained by **processing**, **organizing** and interpreting data.
- Information** helps in **decision-making**.

### Example : From Data to Information

| Data (Raw Facts) | Processing             | Information (Meaningful)  |
|------------------|------------------------|---------------------------|
| 101              | Identified as Roll No. | Roll No. = 101            |
| Amit             | Identified as Name     | Name = Amit               |
| 72.5             | Identified as Marks    | Marks = 72.5              |
| Akola            | Identified as City     | City = Akola              |
| 12-09-2026       | Identified as Date     | Date of Exam = 12-09-2026 |

- Thus, when raw data is processed and given context, it becomes information.

### Difference between Data and Information

| Basis   | Data                                      | Information                               |
|---------|-------------------------------------------|-------------------------------------------|
| Meaning | Raw facts and figures                     | Processed form of data with meaning       |
| Nature  | Unorganized                               | Organized and meaningful                  |
| Purpose | Does not help directly in decision making | Helps in decision making                  |
| Form    | Numbers, text, symbols, images etc.       | Useful and relevant form                  |
| Example | 101, Amit, 72.5                           | Roll No. = 101, Name = Amit, Marks = 72.5 |

### Real Life Example :

- Data** : 85, 90, 78, 92 (marks of a student)
- Processing** : Calculate average
- Information** : Average marks = 86.25
- Use** : It helps in deciding the performance of the student.

## 4. Characteristics of Database

A database has the following important characteristics :

### 1 Self-describing Nature

- A database contains **metadata** (data about data) which describes the structure, type and constraints of data.
- This helps users and DBMS to understand the meaning and format of data.

### 2 Integrated Collection of Data

- Data in a database is related and stored in an integrated manner.
- It avoids isolation of data and promotes **consistency**.

### 3 Persistent Storage

- Data in a database is stored permanently in secondary storage (like hard disk).
- It remains **available even** after the **programs** that created it are terminated.

### 4 Reduced Redundancy

- Duplicate data is minimized.
- The same data is not stored multiple times, which saves storage space and maintains consistency.

### 5 Data Consistency

- Since redundancy is reduced, the **data remains** consistent and accurate.
- Changes made in one place are reflected everywhere.

### 6 Data Sharing

- A database allows multiple users to access and share data at the same time.
- It promotes better coordination among users.



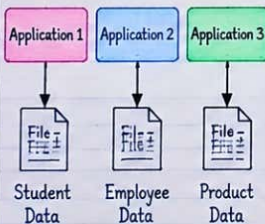
# Need of Database

## 1. Introduction

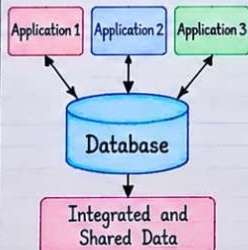
- In the traditional file processing system, data is stored in separate files for each application.
- As the size of data increases and the number of users grow, the file system becomes inefficient.
- To overcome these problems, a database system is needed.
- A database provides a centralized, organized and efficient way to store, manage and retrieve data.

## File System vs Database System

### File System (Traditional)



### Database System



## 2. What is the Need of Database ?

- A database is needed because organizations have to store, manage and process large amounts of data efficiently.
- It helps to overcome the limitations of the file system and provides a better way to handle data.

## 3. Reasons for the Need of Database

### 1 Efficient Data Management

- A database helps to store and manage large amounts of data in an organized manner.
- It provides facilities to easily insert, update, delete and retrieve data.

### 2 Reduce Data Redundancy

- In a file system, the same data is stored in multiple files, which leads to duplication.
- A database stores data in a single place, reducing redundancy and saving storage space.

### 3 Maintain Data Consistency

- When the same data is stored in many files, updating it everywhere is difficult, leading to inconsistency.
- A database maintains a single, consistent copy of data, ensuring accuracy and reliability.

### 4 Data Sharing and Multi-user Access

- A database allows multiple users to access and work with the same data at the same time.
- It provides better coordination and supports data sharing among different departments.

### 5 Data Security

- A database provides security features like user authentication, authorization and access control.
- It protects data from unauthorized access, loss or modification.

### 6 Data Integrity

- A database enforces rules and constraints to maintain the correctness and validity of data.
- It ensures that data is accurate, complete and reliable.

### 7 Easy Data Retrieval

- A database provides powerful query languages (like SQL) to quickly and easily retrieve the required data.
- It supports searching, sorting, filtering and reporting.

### 8 Backup and Recovery

- A database provides facilities for backup and recovery to protect data from system failure or disasters.
- It ensures that data can be restored when needed.

### 9 Support for Transactions

- A database manages multiple operations as a single transaction, ensuring that either all operations are completed or none of them (Atomicity).
- This maintains reliability and prevents data inconsistency.



# Relational Model

## 1. Introduction

- The **Relational Model** is a data model for databases proposed by **Dr. E. F. Codd** in **1970**.
- It **organizes** data into **relations** (tables) consisting of **rows** and **columns**.
- It is based on the mathematical concept of **relation** from **set theory**.
- In this model, data is stored in the form of **tables**, and relationships among data are represented using common attributes.

## 2. What is a Relation ?

- A relation is a **two-dimensional table** with rows and columns.
- It is also called a **table**.
- Each row represents a **tuple** (record) and each column represents an **attribute** (field).
- A relation has a **unique name**.

## 3. Components of a Relation

| Component          | Description                                             |
|--------------------|---------------------------------------------------------|
| <b>Relation</b>    | A table with rows and columns.                          |
| <b>Tuple</b>       | A single row in a relation (represents a record).       |
| <b>Attribute</b>   | A column in a relation (represents a field).            |
| <b>Domain</b>      | Set of possible values for an attribute.                |
| <b>Degree</b>      | Number of attributes (columns) in a relation.           |
| <b>Cardinality</b> | Number of tuples (rows) in a relation.                  |
| <b>Schema</b>      | Structure (name and attributes) of a relation.          |
| <b>Instance</b>    | Actual data present in a relation at a particular time. |

## 4. Example of a Relation

Relation Name : **STUDENT**

| Roll_No | Name  | Class | City     |
|---------|-------|-------|----------|
| 101     | Amina | B.Sc  | Akola    |
| 102     | Yunus | B.Sc  | Balapur  |
| 103     | Sara  | B.Sc  | Amravati |

} Tuple (Row) (Record)

Attribute (Column)

Attribute (Column)

Attribute (Column)

Attribute (Column)

- The above table is a relation named **STUDENT**.
- Each row is a **tuple**.
- Each column is an **attribute**.
- The **domain** of Roll\_No is {101, 102, 103, ...}.
- The **degree** of the relation is 4 (four attributes).
- The **cardinality** is 3 (three tuples).

## 5. Properties of a Relation

- Each cell contains a **single value** (atomic value).
- Each attribute has a **unique name**.
- All values in a column are from the **same domain**.
- The order of **rows** (tuples) is **not important**.
- The order of columns (attributes) is **not important**.
- Duplicate rows** are **not allowed** (each tuple must be unique).
- All rows in a relation have the **same number of columns**.

## 6. Keys in Relational Model

| Key Type             | Description                                                          |
|----------------------|----------------------------------------------------------------------|
| <b>Super Key</b>     | A set of one or more attributes that can uniquely identify a tuple.  |
| <b>Candidate Key</b> | A minimal super key.                                                 |
| <b>Primary Key</b>   | A candidate key chosen to identify tuples uniquely (cannot be NULL). |
| <b>Alternate Key</b> | Candidate keys that are not selected as primary key.                 |
| <b>Foreign Key</b>   | An attribute that references the primary key of another relation.    |

## 7. Domain

- A **domain** is the set of possible values that an attribute can take.
- Example :**  
Domain of Class = {B.Sc, M.Sc, B.A}  
Domain of City = {Akola, Amravati, Pune, Nagpur}

## 8. Relationships in Relational Model

- Two relations can be related using common attributes.
- Relationships are established using **primary key** and **foreign key**.
- Example :**

| STUDENT |       |         | DEPARTMENT |                  |
|---------|-------|---------|------------|------------------|
| Roll_No | Name  | Dept_ID | Dept_ID    | Dept_Name        |
| 101     | Amina | 1       | 1          | Computer Science |
| 102     | Yunus | 2       | 2          | Zoology          |
| 103     | Sara  | 1       |            |                  |

Relationship (Foreign Key)



# Architecture Of DBMS

## 1. Introduction

- Architecture of DBMS refers to the **overall structure** or **design** of a database system.
- It shows how different components of DBMS interact with each other to store, manage, process and retrieve data efficiently.
- The architecture of DBMS is generally based on the **Three Level Architecture** proposed by **ANSI/SPARC**.

## 2. Three Level Architecture

- It divides the database system into **three levels** to provide **data independence**.
- It separates the user's view of data from its actual physical storage.

### 1 External Level (View Level)

- This level is **closest to the users**.
- It shows **only the required part** of data to each user.
- Different users can have **different views** of the same database.
- It **hides** the complex details of data storage.
- Also called **User View**.

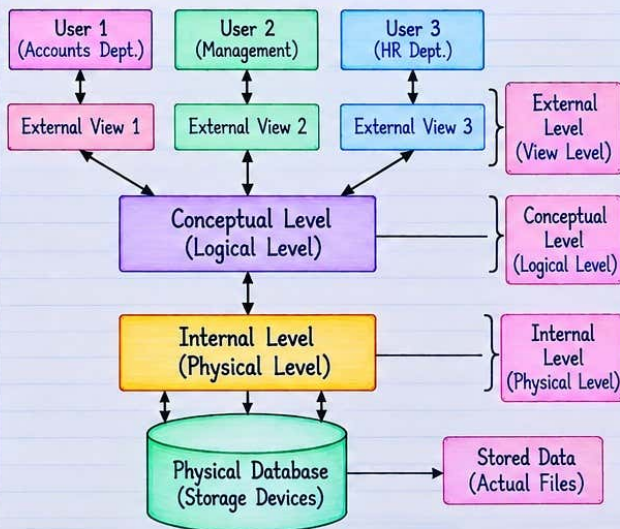
### 2 Conceptual Level (Logical Level)

- This level represents the **overall logical structure** of the database.
- It shows **what data** is stored in the database and **what relationships** exist among data.
- It is **independent of physical storage**.
- Also called **Community View**.

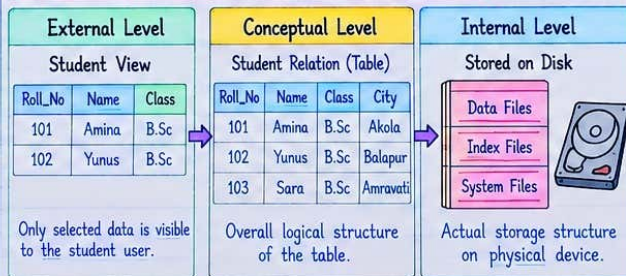
### 3 Internal Level (Physical Level)

- This level describes how data is **actually stored** in the physical storage devices (like hard disk).
- It deals with file organization, indexes, access methods and storage structures.
- Also called **Physical View**.

## 3. Diagram of Three Level Architecture



## 6. Example of Three Level Architecture



## 4. Components of DBMS Architecture

| Component         | Description                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------|
| External Level    | Provides user specific view of data.                                                           |
| Conceptual Level  | Represents overall logical structure of the database.                                          |
| Internal Level    | Describes physical storage structure.                                                          |
| Schema            | Specification of database at each level (External schema, Conceptual schema, Internal schema). |
| Mapping           | Provides connection between different levels (External-Conceptual, Conceptual-Internal).       |
| Data Independence | Ability to change the schema at one level without affecting the next higher level.             |
| DBMS Software     | Manages all these levels and provides interface between users and physical data.               |

## 5. Advantages of Three Level Architecture

- 1 Data **independence** (logical and physical).
- 2 Security and data **abstraction**.
- 3 Multiple user views.
- 4 Reduced complexity.
- 5 Easy **maintenance** and modification.
- 6 Better data **sharing** and integration.

## 7. Conclusion

- The three level architecture of DBMS helps in **data abstraction**, **data independence** and **security**.
- It provides a **clear separation** between user view, logical structure and physical storage of data, making the database system more **flexible** and **efficient**.



# DBA and its Role

## 1. Introduction

- DBA stands for **Database Administrator**.
- A DBA is a specialized person who manages, controls and maintains the database system of an organization.
- The DBA is responsible for ensuring that the database is available, secure, efficient and reliable.
- DBA acts as a bridge between users and the database system.

## 2. Who is a DBA ?

- A Database Administrator (DBA) is a person or a team who is responsible for the **overall management** of the database.
- The DBA has technical and managerial skills to handle database systems.
- He/She ensures that the database runs smoothly and meets the needs of the organization.

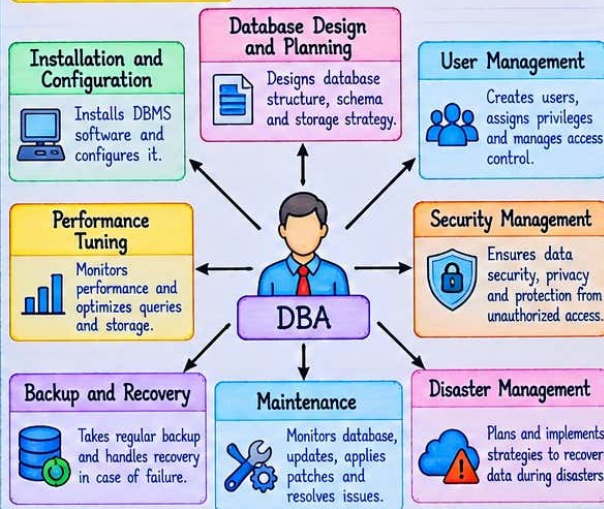
## 3. Main Objectives of a DBA

- To provide high availability of data.
- To maintain data security and integrity.
- To ensure efficient performance.
- To handle user access and permissions.
- To backup and recover the database.
- To solve problems and ensure smooth operation of the system.

## 4. Position of DBA in an Organization

- The DBA works with end users, application developers and system administrators.
- The DBA understands the requirements of users and manages the database accordingly.
- He/She ensures that data is available to authorized users at the right time.

## 5. Role of DBA



## 6. Key Responsibilities of a DBA

| Sr. No. | Responsibility               | Description                                                           |
|---------|------------------------------|-----------------------------------------------------------------------|
| 1       | Database Design              | Creates and maintains schema, tables, indexes and storage structures. |
| 2       | Installation & Configuration | Installs DBMS and configures it as per organizational needs.          |
| 3       | User Management              | Creates users, assigns privileges and controls access.                |
| 4       | Security Management          | Implements security policies, permissions and auditing.               |
| 5       | Backup and Recovery          | Performs regular backup and ensures data recovery.                    |
| 6       | Performance Monitoring       | Monitors performance and optimizes queries.                           |
| 7       | Maintenance                  | Handles upgrades, patches and routine checks.                         |
| 8       | Disaster Management          | Prepares and implements disaster recovery plan.                       |

## 7. DBA Activities

- Creating and modifying database objects (tables, views, indexes etc.)
- Managing user accounts and privileges.
- Monitoring and improving database performance.
- Ensuring data security and integrity.
- Handling backup and recovery operations.
- Managing database storage and space.
- Resolving database issues and errors.
- Planning for disaster recovery.
- Ensuring compliance with organizational policies.

## 8. Skills Required for a DBA

- 1 Good knowledge of DBMS software (Oracle, MySQL, SQL Server etc.).
- 2 Strong SQL and query optimization skills.
- 3 Knowledge of operating systems (Windows/Linux).
- 4 Problem solving and troubleshooting skills.
- 5 Understanding of backup, recovery and security concepts.
- 6 Good communication and management skills.

## 9. Importance of DBA

- Ensures smooth and continuous operation of the database.
- Maintains data security, integrity and consistency.
- Provides efficient data access to authorized users.
- Reduces downtime and prevents data loss.
- Improves performance and reliability of the system.
- Plays a critical role in the success of any organization that relies on data.

## 10. Conclusion

- A Database Administrator (DBA) is an essential part of a database system.
- The DBA manages, controls and maintains the database to ensure its availability, security, performance and reliability.
- Without a DBA, it would be difficult to handle large amounts of data efficiently and safely.
- Thus, the DBA plays a vital role in the successful implementation and use of a database system.



# Structured Query Language (SQL)

## 1. Introduction to SQL

- SQL (Structured Query Language) is a standard language used to communicate with a database.
- It is used to create, manage, retrieve, update and delete data in a relational database management system (RDBMS).
- SQL was developed by IBM in 1970s and standardized by ANSI and ISO.
- It is a non-procedural language, meaning we specify what data is needed, not how to get it.

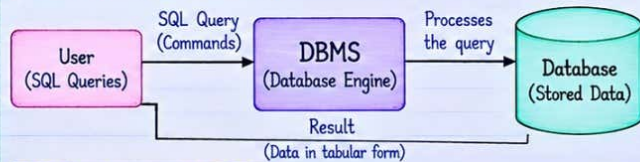
## 2. What is SQL ?

- SQL is a query language used to access and manipulate data in databases.
- It allows users to perform various operations such as creating tables, inserting data, updating data, deleting data and retrieving data.
- SQL is used in many database systems like Oracle, MySQL, SQL Server, PostgreSQL, SQLite etc.

## 3. Purpose of SQL

- To define the structure of the database (tables, indexes, etc.).
- To manipulate data stored in the database.
- To control access to the data.
- To manage transactions.
- To retrieve information from the database easily and efficiently.

## 4. Working of SQL



## 5. Features of SQL

| Sr. No. | Feature                      | Description                                                                            |
|---------|------------------------------|----------------------------------------------------------------------------------------|
| 1       | Simple and Easy              | SQL has simple English-like syntax, easy to learn and use.                             |
| 2       | Standardized Language        | SQL is standardized by ANSI and ISO, so it can be used with many RDBMS software.       |
| 3       | Non-procedural Language      | User specifies <u>what</u> data is needed, not how to retrieve it.                     |
| 4       | Supports Multiple Operations | Used for data definition, data manipulation, data control and transaction control.     |
| 5       | Portable                     | Same SQL commands can be used in different database systems with little or no changes. |
| 6       | Interactive Language         | Allows users to interact with the database directly through queries.                   |
| 7       | Handles Large Amount of Data | Efficiently retrieves and manages large volumes of data.                               |
| 8       | Provides Security            | Supports access control and permissions through DCL commands.                          |
| 9       | Supports Views and Indexes   | Helps in data abstraction (views) and improves performance (indexes).                  |
| 10      | Supports Transactions        | Ensures data consistency using TCL commands (COMMIT, ROLLBACK, SAVEPOINT).             |

## 6. Advantages of SQL

- 1 Easy to learn and use.
- 2 Standardized and widely supported.
- 3 Provides powerful data retrieval capabilities.
- 4 Allows management of large amounts of data.
- 5 Ensures data security and integrity.
- 6 Supports multiple users and applications.
- 7 Works with various database systems.
- 8 Helpful in creating views, indexes and stored objects.
- 9 Non-procedural, so queries are simple and flexible.
- 10 Used for complete database management (create, read, update, delete, control and transaction).

## 7. Disadvantages of SQL

- 1 Requires knowledge of database concepts.
- 2 Handling complex queries can be difficult for beginners.
- 3 Slight variations in syntax between different RDBMS.
- 4 Not suitable for complex procedural logic (for that, PL/SQL, stored procedures etc. are used).
- 5 Performance may decrease with very large and complex queries.

## 8. Applications of SQL

- Used in database applications like banking, hospitals, schools, e-commerce, etc.
- Helps in data analysis and reporting.
- Used in web applications and software development.
- Used in business intelligence and data warehousing.
- Helps organizations to store, manage and analyze data efficiently.



# Data Types in SQL

## 1. Introduction

- Data type specifies the kind of data that can be stored in a column of a table.
- It defines the **format**, **size** and **range** of values that can be stored.
- It helps to maintain **data integrity** by ensuring that only valid data is stored.
- Different **DBMS** (like Oracle, MySQL, SQL Server) have slightly different data types.

## 2. Importance of Data Types

- Helps to store data **efficiently**.
- Ensures data **accuracy** and **consistency**.
- Reduces **storage space**.
- Improves **database performance**.
- Prevents invalid data entry.
- Helps in defining **constraints** and rules.

## 3. Categories of Data Types

SQL data types are broadly classified into :

- Numeric Data Types
- Character / String Data Types
- Date and Time Data Types
- Boolean Data Type
- Binary / Large Object Data Types
- Other Special Data Types

## 4. Numeric Data Types

Used to store numeric values (integers or decimal numbers).

| Data Type     | Description            | Example     | Range / Size                               |
|---------------|------------------------|-------------|--------------------------------------------|
| INT / INTEGER | Stores whole numbers   | 10, -5, 100 | Depends on DBMS (usually 4 bytes)          |
| SMALLINT      | Smaller integer values | 25          | 2 bytes                                    |
| BIGINT        | Large integer values   | 1234567890  | 8 bytes                                    |
| DECIMAL(p, s) | Exact decimal numbers  | 123.45      | p = total digits, s = digits after decimal |
| NUMERIC(p, s) | Same as DECIMAL        | 567.89      | p, s as specified                          |
| FLOAT         | Floating point numbers | 3.14        | 4 or 8 bytes                               |
| REAL          | Single precision float | 6.28        | 4 bytes                                    |
| DOUBLE        | Double precision float | 3.14159     | 8 bytes                                    |

## 5. Character / String Data Types

Used to store alphanumeric data (letters, numbers, symbols).

| Data Type   | Description                            | Example                  | Size / Limit       |
|-------------|----------------------------------------|--------------------------|--------------------|
| CHAR(n)     | Fixed length string                    | 'SQL'                    | n (1 to 2000)      |
| VARCHAR(n)  | Variable length string                 | 'Database'               | n (1 to 4000)      |
| TEXT / CLOB | Large text data                        | 'This is a long text...' | Very large (>4000) |
| NCHAR(n)    | Fixed length Unicode characters        | 'A'                      | n                  |
| NVARCHAR(n) | Variable length Unicode characters     | 'Hello'                  | n                  |
| LONG        | Large character data (Oracle specific) | 'Long text...'           | Up to 2 GB         |

## 6. Date and Time Data Types

Used to store date and time values.

| Data Type             | Description                                  | Example                 | Format / Range         |
|-----------------------|----------------------------------------------|-------------------------|------------------------|
| DATE                  | Stores both date and time                    | 2025-09-30 14:30:00     | YYYY-MM-DD HH:MI:SS    |
| TIME                  | Stores only time                             | 14:30:00                | HH:MI:SS               |
| TIMESTAMP             | Stores date and time with fractional seconds | 2025-09-30 14:30:00.123 | YYYY-MM-DD HH:MI:SS.FF |
| DATETIME (SQL Server) | Stores date and time                         | 2025-09-30 14:30:00     | YYYY-MM-DD HH:MI:SS    |
| INTERVAL (Oracle)     | Stores a time interval                       | INTERVAL '2' DAY        | Day, Hour, Minute etc. |

## 7. Boolean Data Type

Used to store true or false values.

| Data Type | Description           | Example      | Values                            |
|-----------|-----------------------|--------------|-----------------------------------|
| BOOLEAN   | Stores logical values | TRUE / FALSE | TRUE, FALSE (0 or 1 in some DBMS) |

## 8. Binary / Large Object Data Types

Used to store binary data such as images, files, videos, etc.

| Data Type    | Description                 | Example          | Size / Limit |
|--------------|-----------------------------|------------------|--------------|
| BLOB         | Binary Large Object         | Image, file data | Very large   |
| BINARY(n)    | Fixed length binary data    | 1010...          | n bytes      |
| VARBINARY(n) | Variable length binary data | 1010...          | n bytes      |

## 9. Other Special Data Types

| Data Type        | Description                | Example                  | Used In    |
|------------------|----------------------------|--------------------------|------------|
| ENUM (MySQL)     | Predefined set of values   | 'Male', 'Female'         | MySQL      |
| SET (MySQL)      | Multiple predefined values | 'Red', 'Blue'            | MySQL      |
| JSON (MySQL)     | Stores JSON data           | '{"name": "Yunus"}'      | MySQL      |
| XML (SQL Server) | Stores XML data            | '<student>...</student>' | SQL Server |



# Operators in SQL

## 1. Introduction

- Operators are **special symbols** used in SQL to perform operations on data.
- They are used in SQL expressions to **manipulate, filter, compare** and **combine** data.
- Operators can be used in **WHERE clause, SELECT statement**, and other SQL clauses.
- They help to perform **calculations, comparisons, logical operations** and many other tasks in a database.

## 2. Types of Operators in SQL

SQL operators are mainly classified into the following types :

- 1 Arithmetic Operators
- 2 Comparison (Relational) Operators
- 3 Logical Operators
- 4 Bitwise Operators
- 5 Set Operators
- 6 Special Operators

## 3. Arithmetic Operators

Used to perform mathematical operations on numeric data.

| Operator | Meaning             | Example (SQL)                       | Result |
|----------|---------------------|-------------------------------------|--------|
| +        | Addition            | SELECT 10 + 5;                      | 15     |
| -        | Subtraction         | SELECT 10 - 5;                      | 5      |
| *        | Multiplication      | SELECT 10 * 5;                      | 50     |
| /        | Division            | SELECT 10 / 5;                      | 2      |
| % or MOD | Modulus (Remainder) | SELECT 10 % 3;<br>SELECT MOD(10,3); | 1      |

## 4. Comparison (Relational) Operators

Used to compare two values. It returns TRUE, FALSE or UNKNOWN.

| Operator | Meaning                  | Example (SQL)                            |
|----------|--------------------------|------------------------------------------|
| =        | Equal to                 | SELECT * FROM Student WHERE age = 20;    |
| <> or != | Not equal to             | SELECT * FROM Student WHERE age <> 20;   |
| >        | Greater than             | SELECT * FROM Student WHERE marks > 50;  |
| <        | Less than                | SELECT * FROM Student WHERE marks < 50;  |
| >=       | Greater than or equal to | SELECT * FROM Student WHERE marks >= 50; |
| <=       | Less than or equal to    | SELECT * FROM Student WHERE marks <= 50; |

## 5. Logical Operators

Used to combine multiple conditions. It returns TRUE or FALSE.

| Operator | Meaning                                            | Example (SQL)                                        |
|----------|----------------------------------------------------|------------------------------------------------------|
| AND      | Returns TRUE if both conditions are true           | SELECT * FROM Student WHERE age > 18 AND marks > 50; |
| OR       | Returns TRUE if at least one condition is true     | SELECT * FROM Student WHERE age < 18 OR marks > 90;  |
| NOT      | Reverses the result (TRUE to FALSE and vice versa) | SELECT * FROM Student WHERE NOT age = 18;            |

## 6. Bitwise Operators

Used to perform bit-level operations on integer values.

| Operator | Meaning     | Example (SQL)  |
|----------|-------------|----------------|
| &        | Bitwise AND | SELECT 5 & 3;  |
|          | Bitwise OR  | SELECT 5   3;  |
| ^        | Bitwise XOR | SELECT 5 ^ 3;  |
| ~        | Bitwise NOT | SELECT ~5;     |
| <<       | Left Shift  | SELECT 5 << 1; |
| >>       | Right Shift | SELECT 5 >> 1; |

## 7. Set Operators

Used to combine the results of two or more SELECT statements.

| Operator       | Meaning                                              | Example (SQL)                                    |
|----------------|------------------------------------------------------|--------------------------------------------------|
| UNION          | Combines result sets and removes duplicate rows      | SELECT name FROM A UNION SELECT name FROM B;     |
| UNION ALL      | Combines result sets and includes duplicate rows     | SELECT name FROM A UNION ALL SELECT name FROM B; |
| INTERSECT      | Returns only common rows in both result sets         | SELECT name FROM A INTERSECT SELECT name FROM B; |
| MINUS (Oracle) | Returns rows from first query that are not in second | SELECT name FROM A MINUS SELECT name FROM B;     |

## 8. Special Operators

| Operator    | Meaning                               | Example (SQL)                                                                 |
|-------------|---------------------------------------|-------------------------------------------------------------------------------|
| IN          | Checks if a value is in a list        | SELECT * FROM Student WHERE city IN ('Akola', 'Pune');                        |
| BETWEEN     | Checks if a value is in a range       | SELECT * FROM Student WHERE marks BETWEEN 50 AND 90;                          |
| LIKE        | Used for pattern matching             | SELECT * FROM Student WHERE name LIKE 'S%';                                   |
| IS NULL     | Checks for NULL value                 | SELECT * FROM Student WHERE phone IS NULL;                                    |
| IS NOT NULL | Checks for non-NULL value             | SELECT * FROM Student WHERE phone IS NOT NULL;                                |
| EXISTS      | Checks if a subquery returns any rows | SELECT * FROM Student s WHERE EXISTS (SELECT 1 FROM Fee f WHERE f.id = s.id); |

## 9. Precedence of Operators

Operators are evaluated in the following order (from highest to lowest):

| Order | Operator Type          | Operators      |
|-------|------------------------|----------------|
| 1     | Parentheses            | ( )            |
| 2     | Arithmetic             | * / %          |
| 3     | Addition / Subtraction | + -            |
| 4     | Comparison             | = <> > < >= <= |
| 5     | NOT                    | NOT            |
| 6     | AND                    | AND            |
| 7     | OR                     | OR             |

## 10. Conclusion

- Operators in SQL are used to perform various operations on data such as **arithmetic, comparison, logical** and **set operations**.
- They help in **filtering, manipulating** and **combining data** efficiently in database queries.

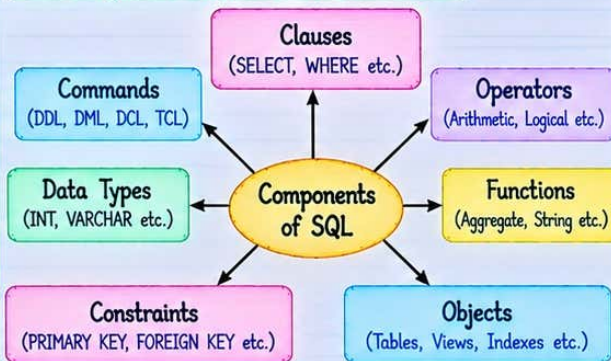


# Components Of SQL

## 1. Introduction

- SQL (Structured Query Language) is a **standard** language used to communicate with a database.
- It is used to define, create, manipulate and control data stored in a **relational** database.
- SQL is made up of different **components** (clauses, commands, operators, functions etc.) that work together to perform various tasks.
- These components help in interacting with the database **easily** and **efficiently**.

## 2. Major Components of SQL



## 3. SQL Commands

SQL commands are used to perform specific tasks.

| Type                                  | Commands                             | Purpose                                              | Example                                         |
|---------------------------------------|--------------------------------------|------------------------------------------------------|-------------------------------------------------|
| DDL<br>(Data Definition Language)     | CREATE<br>ALTER<br>DROP<br>TRUNCATE  | Define and manage database structure (tables, etc.). | CREATE TABLE Student(id INT, name VARCHAR(50)); |
| DML<br>(Data Manipulation Language)   | SELECT<br>INSERT<br>UPDATE<br>DELETE | Retrieve and manipulate data in tables.              | SELECT * FROM Student;                          |
| DCL<br>(Data Control Language)        | GRANT<br>REVOKE                      | Control access and permissions on data.              | GRANT SELECT ON Student TO user1;               |
| TCL<br>(Transaction Control Language) | COMMIT<br>ROLLBACK<br>SAVEPOINT      | Manage transactions in database.                     | COMMIT;<br>ROLLBACK;                            |

## 4. SQL Clauses

Clauses are used with SELECT statement to filter, sort and arrange the data.

| Clause      | Purpose                            | Example                            |
|-------------|------------------------------------|------------------------------------|
| SELECT      | Selects columns from a table       | SELECT name, age FROM Student;     |
| FROM        | Specifies the table name           | FROM Student;                      |
| WHERE       | Filters records based on condition | WHERE age > 18;                    |
| GROUP BY    | Groups rows with same values       | GROUP BY city;                     |
| HAVING      | Filters groups                     | HAVING COUNT(*) > 1;               |
| ORDER BY    | Sorts the result                   | ORDER BY name ASC;                 |
| LIMIT / TOP | Limits number of records           | LIMIT 5;                           |
| DISTINCT    | Returns unique values              | SELECT DISTINCT city FROM Student; |

## 5. SQL Operators

Operators are used to perform operations on data.

| Type       | Operators                       | Example                                                 |
|------------|---------------------------------|---------------------------------------------------------|
| Arithmetic | + - * / %                       | SELECT 10 + 5;                                          |
| Comparison | = < > <= >= <=                  | SELECT * FROM Student WHERE age >= 18;                  |
| Logical    | AND OR NOT                      | SELECT * FROM Student WHERE age > 18 AND city = 'Pune'; |
| Bitwise    | &   ^ ~ << >>                   | SELECT 5 & 3;                                           |
| Set        | UNION UNION ALL INTERSECT MINUS | SELECT name FROM A UNION SELECT name FROM B;            |

## 6. SQL Functions

Functions are built-in methods that perform specific tasks.

| Type                | Functions                              | Example                          |
|---------------------|----------------------------------------|----------------------------------|
| Aggregate Functions | COUNT(), SUM(), AVG(), MAX(), MIN()    | SELECT COUNT(*) FROM Student;    |
| String Functions    | UPPER(), LOWER(), LENGTH(), SUBSTR()   | SELECT UPPER(name) FROM Student; |
| Date Functions      | CURRENT_DATE(), NOW(), YEAR(), MONTH() | SELECT YEAR(NOW());              |
| Numeric Functions   | ROUND(), ABS(), CEIL(), FLOOR()        | SELECT ROUND(3.1416, 2);         |

## 7. Data Types

Data types specify the type of data that can be stored.

| Type        | Data Types                                  | Example           |
|-------------|---------------------------------------------|-------------------|
| Numeric     | INT, SMALLINT, BIGINT, DECIMAL, FLOAT, REAL | age INT           |
| Character   | CHAR(n), VARCHAR(n), TEXT                   | name VARCHAR(50)  |
| Date & Time | DATE, TIME, DATETIME, TIMESTAMP             | dob DATE          |
| Boolean     | BOOLEAN                                     | is_active BOOLEAN |
| Binary      | BLOB, BINARY, VARBINARY                     | image BLOB        |

## 8. Constraints

Constraints are used to enforce rules on data.

| Constraint  | Purpose                               | Example                             |
|-------------|---------------------------------------|-------------------------------------|
| PRIMARY KEY | Uniquely identifies each record       | id INT PRIMARY KEY                  |
| FOREIGN KEY | Maintains relationship between tables | dept_id INT REFERENCES Dept(id)     |
| UNIQUE      | Ensures all values are unique         | email VARCHAR(50) UNIQUE            |
| NOT NULL    | Prevents NULL values                  | name VARCHAR(50) NOT NULL           |
| CHECK       | Checks condition on values            | age INT CHECK (age >= 0)            |
| DEFAULT     | Provides a default value              | status VARCHAR(10) DEFAULT 'Active' |

## 9. Other Components

Apart from commands, clauses and operators, SQL also includes objects and views.

| Component         | Description                      | Example                                                             |
|-------------------|----------------------------------|---------------------------------------------------------------------|
| Tables            | Store data in rows and columns   | CREATE TABLE Student(...);                                          |
| Views             | Virtual table based on a query   | CREATE VIEW Adult_Student AS SELECT * FROM Student WHERE age >= 18; |
| Indexes           | Improve search performance       | CREATE INDEX idx_name ON Student(name);                             |
| Sequences         | Generate unique numbers (Oracle) | CREATE SEQUENCE seq_id START WITH 1;                                |
| Stored Procedures | Set of SQL statements (PL/SQL)   | CREATE PROCEDURE GetStudents AS ...;                                |



# Data Definition Language (DDL)

## 1. Introduction to DDL

- DDL (Data Definition Language) is a subset of SQL used to define and manage the **structure** of database objects such as tables, views, indexes, etc.
- It is used to **create, modify** and **delete** database objects.
- DDL commands work on the **structure** of the database, not on the data inside the tables.
- DDL statements are **auto-committed** in most databases (cannot be rolled back).
- Examples: **CREATE, ALTER, DROP, TRUNCATE, RENAME**.

## 2. DDL Commands

| Command         | Purpose                                                   |
|-----------------|-----------------------------------------------------------|
| <b>CREATE</b>   | Creates a new database object (table, view, index etc.).  |
| <b>ALTER</b>    | Modifies the structure of an existing object.             |
| <b>DROP</b>     | Deletes an existing object permanently.                   |
| <b>TRUNCATE</b> | Removes all records from a table (but structure remains). |
| <b>RENAME</b>   | Changes the name of an object.                            |

## 3. CREATE Command

- Used to create a new table, view, index or database.
- Syntax (Table Creation):

```
CREATE TABLE table_name (
    column1 datatype(size) constraint,
    column2 datatype(size) constraint,
    ...
);
```

### Common Data Types:

- INT - Integer
- VARCHAR(n) - Variable string
- CHAR(n) - Fixed string
- DATE - Date
- FLOAT - Floating point
- DECIMAL(n,m) - Decimal number

Example: Create a table Student

```
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    course VARCHAR(30)
);
```

## 5. DROP Command

- Used to delete an existing table or other database object permanently.
- Syntax:

**DROP TABLE** table\_name;

Example:

**DROP TABLE** Student;

### Note:

- Table structure and data both are deleted.
- It cannot be recovered easily.

## 8. Complete Example

Performing different DDL operations on a table.

| Step              | Command                                                                                  | Description                                        |
|-------------------|------------------------------------------------------------------------------------------|----------------------------------------------------|
| 1. Create table   | <b>CREATE TABLE</b> Employee ( emp_id INT PRIMARY KEY, name VARCHAR(50), salary FLOAT ); | Creates a new table 'Employee' with three columns. |
| 2. Add column     | <b>ALTER TABLE</b> Employee <b>ADD</b> department VARCHAR(30);                           | Adds a new column.                                 |
| 3. Modify column  | <b>ALTER TABLE</b> Employee <b>MODIFY</b> salary DECIMAL(10,2);                          | Modifies the data type of salary column.           |
| 4. Drop column    | <b>ALTER TABLE</b> Employee <b>DROP COLUMN</b> department;                               | Deletes the department column.                     |
| 5. Truncate table | <b>TRUNCATE TABLE</b> Employee;                                                          | Removes all records.                               |
| 6. Drop table     | <b>DROP TABLE</b> Employee;                                                              | Deletes the table completely.                      |

## 4. ALTER Command

- Used to modify the structure of an existing table.
- It can be used to add, modify or delete columns and constraints.

| Operation                 | Syntax                                                                          | Example                                                                          |
|---------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Add Column                | <b>ALTER TABLE</b> table_name <b>ADD</b> column_name datatype;                  | <b>ALTER TABLE</b> Student <b>ADD</b> address VARCHAR(50);                       |
| Modify Column (Data Type) | <b>ALTER TABLE</b> table_name <b>MODIFY</b> column_name datatype;               | <b>ALTER TABLE</b> Student <b>MODIFY</b> age INT;                                |
| Drop Column               | <b>ALTER TABLE</b> table_name <b>DROP COLUMN</b> column_name;                   | <b>ALTER TABLE</b> Student <b>DROP COLUMN</b> address;                           |
| Add Constraint            | <b>ALTER TABLE</b> table_name <b>ADD CONSTRAINT</b> constraint_name constraint; | <b>ALTER TABLE</b> Student <b>ADD CONSTRAINT</b> chk_age <b>CHECK</b> (age > 0); |

## 6. TRUNCATE Command

- Used to remove all records from a table. Table structure remains the same.
- It is faster than **DELETE** because it deallocates data pages.
- Syntax:

**TRUNCATE TABLE** table\_name;

Example:

**TRUNCATE TABLE** Student;

## 7. RENAME Command

- Used to change the name of a table or other object.
- Syntax:

**RENAME** old\_name TO new\_name;

Example:

**RENAME** Student TO Student\_Info;

## 9. Important Points

- DDL commands are used to define and manage the structure of database objects.
- They are **auto-committed** (cannot be rolled back in most DBMS).
- CREATE** creates a new object.
- ALTER** modifies an existing object.
- DROP** deletes an object permanently.
- TRUNCATE** removes all records but keeps structure.
- RENAME** changes the name of an object.
- DDL does not affect the data inside the table (except **TRUNCATE** which deletes all data).
- Constraints like **PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL** can also be defined using DDL.
- DDL commands are supported in all major RDBMS (Oracle, MySQL, SQL Server, PostgreSQL etc.).

## 10. Conclusion

- DDL is an important part of SQL used to **create, modify** and **delete** the structure of database objects.
- It helps in maintaining the database **schema** according to the requirements.
- The main DDL commands are **CREATE, ALTER, DROP, TRUNCATE** and **RENAME**.
- These commands make it easy to define and manage the structure of a database efficiently.



# Data Manipulation Language (DML)

## 1. Introduction to DML

- DML (Data Manipulation Language) is a subset of SQL used to manipulate the data stored in the database.
- It is used to **insert**, **update**, **delete** and retrieve data from the tables.
- DML commands work on the **data inside the tables** (not on structure).
- DML changes the data and these changes are permanently or temporarily stored depending on the transaction.
- Main DML commands are **SELECT**, **INSERT**, **UPDATE**, **DELETE**.

## 2. DML Commands

| Command       | Purpose                                 |
|---------------|-----------------------------------------|
| <b>SELECT</b> | Retrieves data from one or more tables. |
| <b>INSERT</b> | Adds new records into a table.          |
| <b>UPDATE</b> | Modifies existing records in a table.   |
| <b>DELETE</b> | Removes existing records from a table.  |

## 3. General Syntax

|                                                                                     |
|-------------------------------------------------------------------------------------|
| <b>SELECT</b> column1, column2 <b>FROM</b> table_name<br>[WHERE condition];         |
| <b>INSERT INTO</b> table_name (column1, column2)<br><b>VALUES</b> (value1, value2); |
| <b>UPDATE</b> table_name<br><b>SET</b> column1 = value1<br>[WHERE condition];       |
| <b>DELETE FROM</b> table_name<br>[WHERE condition];                                 |

## 4. SELECT Command

- Used to retrieve data from a table.
- It can be used to display all columns or selected columns.
- Syntax:

```
SELECT column1, column2 FROM table_name
[WHERE condition];
```

Example 1: Display all records

```
SELECT * FROM Student;
```

Result:

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 21  | B.Sc   |
| 3  | Aman | 20  | BCA    |
| 4  | Neha | 22  | B.Sc   |

Example 2: Display selected columns

```
SELECT name, age FROM Student;
```

Result:

| name | age |
|------|-----|
| Ali  | 20  |
| Sara | 21  |
| Aman | 20  |
| Neha | 22  |

Example 3: Using WHERE clause

```
SELECT * FROM Student WHERE age > 20;
```

Result:

| id | name | age | course |
|----|------|-----|--------|
| 2  | Sara | 21  | B.Sc   |
| 4  | Neha | 22  | B.Sc   |

## 6. UPDATE Command

- Used to modify existing data in a table.
- It updates all records that satisfy the condition.
- Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
[WHERE condition];
```

Example 1: Update a single column

```
UPDATE Student
SET age = 23
WHERE name = 'Ali';
```

Example 2: Update multiple columns

```
UPDATE Student
SET age = 21, course = 'M.Sc'
WHERE id = 2;
```

## 8. Important Points

- DML commands work on data, not on table structure.
- SELECT is used to retrieve data.
- INSERT adds new records.
- UPDATE updates existing records.
- DELETE removes records.
- DELETE removes records.
- DML changes are permanent unless a transaction (ROLLBACK) is used.
- DML commands are used frequently in real-time applications.

## 5. INSERT Command

- Used to insert new records into a table.
- Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example 1: Insert a single record

```
INSERT INTO Student (id, name, age, course)
VALUES (5, 'Zara', 21, 'BCA');
```

Example 2: Insert only selected columns

```
INSERT INTO Student (name, age)
VALUES ('Rahul', 20);
```

Example 3: Insert multiple records

```
INSERT INTO Student (id, name, age, course)
VALUES (6, 'Sameer', 21, 'B.Sc'),
(7, 'Ayesha', 22, 'BCA');
```

Before Insert:

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 21  | B.Sc   |
| 3  | Aman | 20  | BCA    |
| 4  | Neha | 22  | B.Sc   |

After Insert:

| id | name   | age | course |
|----|--------|-----|--------|
| 1  | Ali    | 20  | B.Sc   |
| 2  | Sara   | 21  | B.Sc   |
| 3  | Aman   | 20  | BCA    |
| 4  | Neha   | 22  | B.Sc   |
| 5  | Zara   | 21  | BCA    |
| 6  | Sameer | 21  | B.Sc   |
| 7  | Ayesha | 22  | BCA    |

## 7. DELETE Command

- Used to remove existing records from a table.
- It permanently deletes the records that satisfy the condition.
- Syntax:

```
DELETE FROM table_name
[WHERE condition];
```

Example 1: Delete a specific record

```
DELETE FROM Student
WHERE id = 3;
```

Example 2: Delete all records (use with care)

```
DELETE FROM Student;
```

Before Delete:

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 23  | B.Sc   |
| 2  | Sara | 21  | B.Sc   |
| 3  | Aman | 20  | BCA    |
| 4  | Neha | 22  | B.Sc   |

After Delete (id = 3):

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 23  | B.Sc   |
| 2  | Sara | 21  | B.Sc   |
| 4  | Neha | 22  | B.Sc   |

## 9. Conclusion

- DML is an essential part of SQL used to manipulate the data in database tables.
- The main DML commands are **SELECT**, **INSERT**, **UPDATE** and **DELETE**.
- These commands help in adding, retrieving, modifying and deleting data.
- They provide easy and efficient way to manage the data stored in a relational database.
- DML commands are widely used in real-world database applications.



# Data Control Language (DCL)

## 1. Introduction to DCL

- DCL (Data Control Language) is a subset of SQL used to control access to the data stored in a database.
- It is used to **grant** and **revoke** permissions to users or roles.
- DCL commands help maintain data **security**, **integrity** and **authorization**.
- It controls who can access the database objects (tables, views, procedures, etc.) and what operations they can perform.
- Main DCL commands are **GRANT** and **REVOKE**.

## 2. Purpose of DCL

- To provide **controlled access** to database objects.
- To prevent **unauthorized access** to data.
- To maintain **security** and **privacy** of information.
- To assign or **remove privileges** for specific users or roles.
- To ensure only authorized users can perform certain operations (**SELECT**, **INSERT**, **UPDATE**, etc.).

## 3. DCL Commands

| Command       | Purpose                                                                        |
|---------------|--------------------------------------------------------------------------------|
| <b>GRANT</b>  | Gives specific privileges (permissions) on database objects to a user or role. |
| <b>REVOKE</b> | Removes previously granted privileges from a user or role.                     |

## 4. Privileges in DCL

Different types of privileges that can be granted:

| Privilege         | Description                                         |
|-------------------|-----------------------------------------------------|
| <b>SELECT</b>     | Allows user to read data from a table.              |
| <b>INSERT</b>     | Allows user to add new data.                        |
| <b>UPDATE</b>     | Allows user to modify existing data.                |
| <b>DELETE</b>     | Allows user to remove data.                         |
| <b>ALL</b>        | Grants all privileges on the object.                |
| <b>REFERENCES</b> | Allows creating foreign key reference.              |
| <b>EXECUTE</b>    | Allows execution of stored procedures or functions. |
| <b>USAGE</b>      | Allows use of database objects (e.g. schemas).      |

## 5. GRANT Command

- Used to give specific privileges to a user or role.
- Syntax:

```
GRANT privilege_name (column_name, ...)
ON object_name
TO user_name;
```

## 6. Examples of GRANT Command

### Example 1: Grant SELECT privilege

```
GRANT SELECT
ON Student
TO user1;
```

This allows user1 to read data from Student table.

### Example 2: Grant multiple privileges

```
GRANT INSERT, UPDATE, DELETE
ON Student
TO user2;
```

This allows user2 to insert, update and delete data in Student table.

### Example 3: Grant all privileges

```
GRANT ALL
ON Student
TO user3;
```

This allows user3 to perform all operations on Student table.

### Example 4: Grant privilege on specific column

```
GRANT UPDATE (age, course)
ON Student
TO user4;
```

This allows user4 to update only age and course columns.

## 7. REVOKE Command

- Used to remove previously granted privileges from a user or role.
- Syntax:

```
REVOKE privilege_name (column_name, ...)
ON object_name
FROM user_name;
```

## 8. Examples of REVOKE Command

### Example 1: Revoke SELECT privilege

```
REVOKE SELECT
ON Student
FROM user1;
```

This removes read access from user1.

### Example 2: Revoke multiple privileges

```
REVOKE INSERT, UPDATE
ON Student
FROM user2;
```

This removes insert and update privileges from user2.

### Example 3: Revoke all privileges

```
REVOKE ALL
ON Student
FROM user3;
```

This removes all privileges from user3.

### Example 4: Revoke privilege on column

```
REVOKE UPDATE (age)
ON Student
FROM user4;
```

This removes update access on age column from user4.

## 9. Practical Example

Consider a Student table. We grant and revoke permissions to different users.

| Step | Command                                           | Description                          |
|------|---------------------------------------------------|--------------------------------------|
| 1    | <b>GRANT</b> SELECT ON Student TO user1;          | Allows user1 to read data.           |
| 2    | <b>GRANT</b> INSERT ON Student TO user2;          | Allows user2 to insert data.         |
| 3    | <b>GRANT</b> UPDATE (age) ON Student TO user3;    | Allows user3 to update age column.   |
| 4    | <b>REVOKE</b> SELECT ON Student FROM user1;       | Removes read access from user1.      |
| 5    | <b>REVOKE</b> INSERT ON Student FROM user2;       | Removes insert access from user2.    |
| 6    | <b>REVOKE</b> UPDATE (age) ON Student FROM user3; | Removes update access on age column. |

## 10. Example with Roles

- Privileges can also be given to roles (group of users).

### Example :

```
GRANT SELECT, INSERT ON Student
TO role_student;
```

```
GRANT role_student TO user1;
```

Here, privileges are given to the role 'role\_student' and then the role is assigned to user1.

## 11. Important Points

- DCL commands are used to control access to database objects.
- GRANT** gives privileges and **REVOKE** removes privileges.
- Privileges can be given to users or roles.
- Common privileges: **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **ALL**.
- DCL commands help in maintaining security and data integrity.
- Only users with administrative rights (e.g., DBA) can use DCL commands.

## 12. Conclusion

- DCL is an important part of SQL used to manage access control.
- It includes two main commands: **GRANT** and **REVOKE**.
- It helps in providing security, privacy and controlled access to the data in the database.
- Proper use of DCL ensures that only authorized users can perform specific operations on database objects.



# Transaction Control Language (TCL)

## 1. Introduction to TCL

- TCL (Transaction Control Language) is a subset of SQL used to manage transactions in a database.
- It is used to control the changes made by DML statements (INSERT, UPDATE, DELETE) in a database.
- TCL commands help to maintain data integrity by allowing us to commit, rollback or save the changes.
- The changes made using DML are temporary and can be permanently saved or undone using TCL commands.
- Main TCL commands are COMMIT, ROLLBACK, SAVEPOINT and SET TRANSACTION.

## 5. ROLLBACK Command

- Used to undo the changes made in the current transaction.
- It discards all changes made after the last COMMIT or SAVEPOINT.
- Syntax:

**ROLLBACK;**

**Example :**

```
UPDATE Student SET age = 25
WHERE id = 1;
ROLLBACK;
```

**Result :**

- The change in age is not saved.
- The table remains in its previous state.

## 9. Result Illustration

| Operation      | After Operation (Student Table) |      |     |   | Explanation |                                     |
|----------------|---------------------------------|------|-----|---|-------------|-------------------------------------|
| Initial Data   | id                              | name | Ali | 1 | age 20      | Original data.                      |
| After Update   | 1                               | name | Ali | 1 | age 22      | Change is temporary (until COMMIT). |
| After ROLLBACK | id                              | name | Ali | 1 | age 20      | Change is undone.                   |
| After COMMIT   | d                               | name | Ali | 1 | age 25      | Change is permanently saved.        |

## 2. What is a Transaction ?

- A transaction is a sequence of one or more SQL statements treated as a single unit of work.
- It follows either ACID properties or none of the changes are applied.
- A transaction starts with the first DML statement and ends with COMMIT or ROLLBACK.

**Example of a Transaction :**

```
UPDATE Account SET balance = balance - 500
WHERE id = 1;
UPDATE Account SET balance = balance + 500
WHERE id = 2;
COMMIT;
```

Both updates are treated as a single transaction.

## 6. SAVEPOINT Command

- Used to set a point within a transaction so that we can rollback to that point without undoing the entire transaction.
- Syntax:

**SAVEPOINT** savepoint\_name;

**Example :**

```
UPDATE Student SET age = 22 WHERE id = 1;
SAVEPOINT sp1;
UPDATE Student SET age = 25 WHERE id = 1;
ROLLBACK TO sp1;
```

**Result :**

- The second update is undone.
- The age value is returned to the state at savepoint sp1.

## 3. TCL Commands

| Command         | Purpose                                                               |
|-----------------|-----------------------------------------------------------------------|
| COMMIT          | Permanently saves the changes made in the current transaction.        |
| ROLLBACK        | Undoes the changes made in the current transaction.                   |
| SAVEPOINT       | Sets a point within a transaction to which we can rollback.           |
| SET TRANSACTION | Specifies characteristics of the transaction (e.g., isolation level). |

## 7. SET TRANSACTION Command

- Used to define the characteristics of a transaction such as isolation level, access mode, etc.
- It is used at the beginning of a transaction.
- Syntax:

**SET TRANSACTION**  
[ISOLATION LEVEL level]  
[READ ONLY | READ WRITE];

**Example :**

```
SET TRANSACTION ISOLATION LEVEL
SERIALIZABLE;
SET TRANSACTION READ WRITE;
```

**Result :**

- Defines the transaction behavior for the current session.

## 4. COMMIT Command

- Used to permanently save all changes made in the current transaction.
- After COMMIT, changes cannot be undone.
- Syntax:

**COMMIT;**

**Example :**

```
UPDATE Student SET age = 22
WHERE id = 1;
COMMIT;
```

**Result :**

- The change in age is permanently saved in the Student table.
- It cannot be rolled back.

## 8. Example: Complete Transaction Flow

| Step | SQL Statement                                      | Explanation                                  |
|------|----------------------------------------------------|----------------------------------------------|
| 1    | START TRANSACTION (or first DML statement)         | Transaction begins implicitly.               |
| 2    | UPDATE Student SET age = 22 WHERE id = 1;          | First change (temporary).                    |
| 3    | SAVEPOINT sp1;                                     | Savepoint created.                           |
| 4    | UPDATE Student SET age = 25 WHERE id = 1;          | Second change.                               |
| 5    | ROLLBACK TO sp1;                                   | Undo second change (returns to sp1).         |
| 6    | INSERT INTO Student (id, name) VALUES (2, 'Sara'); | Another change.                              |
| 7    | COMMIT;                                            | All remaining changes are permanently saved. |

## 10. Important Points

- TCL works on data (transactions), not on structure.
- Changes made by DML are temporary until COMMIT.
- ROLLBACK undoes the changes.
- SAVEPOINT allows partial rollback.
- COMMIT permanently saves the changes.
- SET TRANSACTION defines transaction properties.
- TCL helps maintain data consistency and integrity.

## 11. ACID Properties

| Property    | Meaning                                                          |
|-------------|------------------------------------------------------------------|
| Atomicity   | All operations in a transaction are completed or none.           |
| Consistency | Transaction brings the database from one valid state to another. |
| Isolation   | Transactions are isolated from each other.                       |
| Durability  | Once committed, changes are permanent even in case of failure.   |

## 12. Conclusion

- TCL is an essential part of SQL for managing transactions.
- Main commands are COMMIT, ROLLBACK, SAVEPOINT and SET TRANSACTION.
- It helps to maintain data integrity, consistency and reliability.
- Proper use of TCL ensures that the database remains in a correct and stable state.



# SELECT Statement with Clauses

## 1. SELECT Statement : Introduction

- SELECT statement is used to retrieve data from one or more tables in a database.
- It allows us to specify the columns to be displayed and apply different clauses to filter, sort, group and analyze the data.
- Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
[WHERE condition]  
[GROUP BY column]  
[HAVING condition]  
[ORDER BY column [ASC | DESC]];
```

## 2. Basic SELECT Statement

- Used to display specific columns from a table.
- If we want all columns, we use \* (asterisk).
- Syntax:

```
SELECT column1, column2, ... FROM table_name;
```

### Example 1: Display specific columns

```
SELECT id, name, course  
FROM Student;
```

#### Result :

| id | name | course |
|----|------|--------|
| 1  | Ali  | B.Sc   |
| 2  | Sara | BCA    |
| 3  | Aman | B.Sc   |
| 4  | Neha | BCA    |

### Example 2: Display all columns

```
SELECT * FROM Student;
```

#### Result :

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 21  | BCA    |
| 3  | Aman | 20  | B.Sc   |
| 4  | Neha | 22  | BCA    |

## 3. WHERE Clause

- Used to filter records based on a condition.
- It returns only those rows that satisfy the condition.
- Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

### Example : Students with age > 20

```
SELECT id, name, age, course  
FROM Student  
WHERE age > 20;
```

#### Result :

| id | name | age | course |
|----|------|-----|--------|
| 2  | Sara | 21  | BCA    |
| 4  | Neha | 22  | BCA    |

### Common Conditions in WHERE Clause

= Equal to  
<> Not equal to  
> Greater than  
< Less than  
>= Greater than equal to  
<= Less than equal to  
BETWEEN Within a range  
IN Multiple values  
LIKE Pattern matching  
IS NULL Null values

## 4. ORDER BY Clause

- Used to sort the result set in ascending (ASC) or descending (DESC) order.
- By default, it sorts in ascending order.
- Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column_name [ASC | DESC];
```

### Example 1: Sort by name (Ascending)

```
SELECT id, name, age  
FROM Student  
ORDER BY name ASC;
```

#### Result :

| id | name | age |
|----|------|-----|
| 1  | Ali  | 20  |
| 3  | Aman | 20  |
| 4  | Neha | 22  |
| 2  | Sara | 21  |

### Example 2: Sort by age (Descending)

```
SELECT id, name, age  
FROM Student  
ORDER BY age DESC;
```

#### Result :

| id | name | age |
|----|------|-----|
| 4  | Neha | 22  |
| 2  | Sara | 21  |
| 1  | Ali  | 20  |
| 3  | Aman | 20  |

## 5. GROUP BY Clause

- Used to group rows that have the same values in a specific column.
- It is often used with aggregate functions like COUNT(), SUM(), AVG(), etc.
- Syntax:

```
SELECT column_name, aggregate_function(column_name)  
FROM table_name  
GROUP BY column_name;
```

### Example : Count students in each course

```
SELECT course, COUNT(*) AS total_students  
FROM Student  
GROUP BY course;
```

#### Result :

| course | total_students |
|--------|----------------|
| B.Sc   | 2              |
| BCA    | 2              |

## 6. HAVING Clause

- Used to filter grouped records based on a condition.
- It is similar to WHERE, but WHERE is used before grouping and HAVING is used after grouping.
- Syntax:

```
SELECT column_name, aggregate_function(column_name)  
FROM table_name  
GROUP BY column_name  
HAVING condition;
```

### Example : Courses with more than 1 student

```
SELECT course, COUNT(*) AS total_students  
FROM Student  
GROUP BY course  
HAVING COUNT(*) > 1;
```

#### Result :

| course | total_students |
|--------|----------------|
| B.Sc   | 2              |
| BCA    | 2              |

### Difference between WHERE and HAVING

- WHERE filters individual rows before grouping.
- HAVING filters grouped rows after grouping.
- WHERE cannot be used with aggregate functions.
- HAVING is used with aggregate functions (COUNT, SUM, AVG, etc.).

## 7. Using Multiple Clauses Together

We can use WHERE, GROUP BY, HAVING and ORDER BY in a single query.

**Example :** Find courses with more than 1 student and sort by total count

```
SELECT course, COUNT(*) AS total_students  
FROM Student  
WHERE age >= 20  
GROUP BY course  
HAVING COUNT(*) > 1  
ORDER BY total_students DESC;
```

#### Result :

| course | total_students |
|--------|----------------|
| B.Sc   | 2              |
| BCA    | 2              |

## 8. Sample Table Used

Table Name : **Student**

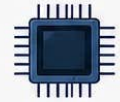
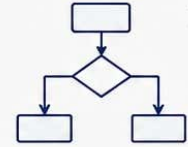
| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 21  | BCA    |
| 3  | Aman | 20  | B.Sc   |
| 4  | Neha | 22  | BCA    |

## 9. Important Points

- 1 SELECT is used to retrieve data from a table.
- 2 WHERE filters rows before grouping.
- 3 GROUP BY groups rows with the same values.
- 4 HAVING filters grouped rows.
- 5 ORDER BY sorts the final result set.
- 6 We can use multiple clauses together to get meaningful results.
- 7 Aggregate functions (COUNT, SUM, AVG, MIN, MAX) are often used with GROUP BY and HAVING.



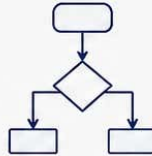
1010101010  
0101010101  
1010101010  
0101010101



# Unit - II



10110  
01001  
10101





# Constraints in SQL (RDBMS)

## 1. Introduction to Constraints

- Constraints are rules applied on columns of a table to restrict the type of data that can be inserted, updated or deleted.
- They help maintain data integrity, consistency and accuracy.
- Constraints are enforced by the DBMS.
- They prevent invalid data from entering the database.
- Constraints can be defined at the time of table creation or after the table is created using ALTER TABLE.
- Main constraints in SQL are: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK and DEFAULT.

## 2. Types of Constraints

| Constraint  | Purpose                                                             |
|-------------|---------------------------------------------------------------------|
| NOT NULL    | Ensures that a column cannot have NULL value.                       |
| UNIQUE      | Ensures that all values in a column are different.                  |
| PRIMARY KEY | Uniquely identifies each record. (Combination of NOT NULL + UNIQUE) |
| FOREIGN KEY | Maintains referential integrity between two tables.                 |
| CHECK       | Ensures that the values satisfy a specific condition.               |
| DEFAULT     | Provides a default value when no value is supplied.                 |

## 3. NOT NULL Constraint

- Ensures that a column cannot have a NULL value.
- Every record must contain a value in this column.

Syntax:

```
CREATE TABLE Student (  
  id INT NOT NULL,  
  name VARCHAR(50) NOT NULL,  
);
```

Example:

```
CREATE TABLE Student (  
  id INT NOT NULL,  
  name VARCHAR(50) NOT NULL,  
  age INT  
);
```

## 4. UNIQUE Constraint

- Ensures that all values in a column are unique (no duplicates).
- A table can have multiple UNIQUE constraints.
- Syntax:

```
CREATE TABLE Student (  
  email VARCHAR(50) UNIQUE  
);
```

Example:

```
CREATE TABLE Student (  
  id INT,  
  email VARCHAR(50) UNIQUE  
);
```

Result:

| id | email          |
|----|----------------|
| 1  | ali@gmail.com  |
| 2  | sara@gmail.com |
| 3  | aman@gmail.com |

If we try to insert a duplicate email, it will give an error.

## 5. PRIMARY KEY Constraint

- Uniquely identifies each record in a table.
- Cannot contain NULL values.
- Only one PRIMARY KEY per table.
- It is a combination of NOT NULL and UNIQUE.
- Syntax:

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,  
  name VARCHAR(50)  
);
```

Example:

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);
```

Id must be unique and cannot be NULL.

Result:

| id | name | age |
|----|------|-----|
| 1  | Ali  | 20  |
| 2  | Sara | 21  |
| 3  | Aman | 20  |

## 6. FOREIGN KEY Constraint

- Maintains a relationship between two tables.
- The value in the child table must exist in the parent table.
- It ensures referential integrity.
- Syntax:

```
CREATE TABLE ChildTable (  
  column_name datatype,  
  FOREIGN KEY (column_name)  
  REFERENCES ParentTable(column_name)  
);
```

Example:

```
CREATE TABLE Department (  
  dept_id INT PRIMARY KEY,  
  dept_name VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES  
  Department(dept_id)  
);
```

Parent Table : Department

| dept_id | dept_name |
|---------|-----------|
| 1       | B.Sc      |
| 2       | BCA       |
| 3       | B.Com     |

Child Table : Student

| id | name | dept_id |
|----|------|---------|
| 1  | Ali  | 1       |
| 2  | Sara | 2       |
| 3  | Aman | 1       |

We cannot insert a dept\_id in Student table if it does not exist in Department table.

## 7. CHECK Constraint

- Ensures that the values in a column satisfy a specific condition.
- It is used to restrict invalid data.
- Syntax:

```
CREATE TABLE Student (  
  age INT CHECK (age >= 18)  
);
```

Example:

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT CHECK (age >= 18)  
);
```

Result:

| id | name | age |
|----|------|-----|
| 1  | Ali  | 20  |
| 2  | Sara | 19  |
| 3  | Aman | 22  |

If we try to insert age less than 18, it will give an error.

## 8. DEFAULT Constraint

- Provides a default value when no value is supplied for a column.
- It is useful to avoid NULL values.
- Syntax:

```
CREATE TABLE Student (  
  course VARCHAR(30) DEFAULT 'B.Sc'  
);
```

Example:

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  course VARCHAR(30) DEFAULT 'B.Sc'  
);
```

Result:

| id | name | course |
|----|------|--------|
| 1  | Ali  | B.Sc   |
| 2  | Sara | B.Sc   |
| 3  | Aman | BCA    |

If no value is provided for course, it will automatically take 'B.Sc'.

## 9. Adding Constraints using ALTER TABLE

- Constraints can also be added after table creation using ALTER TABLE.
- Syntax:

- Add NOT NULL  
ALTER TABLE table\_name  
MODIFY column\_name datatype NOT NULL;
- Add UNIQUE  
ALTER TABLE table\_name  
ADD UNIQUE (column\_name);
- Add PRIMARY KEY  
ALTER TABLE table\_name  
ADD PRIMARY KEY (column\_name);

- Add FOREIGN KEY  
ALTER TABLE table\_name  
ADD FOREIGN KEY (column\_name)  
REFERENCES parent\_table(column\_name);
- Add CHECK  
ALTER TABLE table\_name  
ADD CHECK (condition);
- Add DEFAULT  
ALTER TABLE table\_name  
MODIFY column\_name datatype  
DEFAULT value;

## 10. Important Points

- Constraints maintain data integrity.
- PRIMARY KEY automatically enforces NOT NULL and UNIQUE.
- FOREIGN KEY ensures referential integrity.
- CHECK is used to restrict invalid data.
- DEFAULT provides a default value.
- UNIQUE allows unique values in a column.
- We can add or remove constraints using ALTER TABLE.
- Constraints help maintain consistency and reliability of data in a database.



# Primary Key Constraint

## 1. Introduction

- Primary Key is a constraint used to **uniquely** identify each record in a table.
- It ensures that no two rows have the same value in the primary key column.
- It does not allow **NULL** values.
- A table can have only one primary key, but the primary key can consist of **single column** or **multiple columns** (composite primary key).
- Primary key helps to maintain **data integrity** and ensures that each record is unique.

## 5. Inserting Data

- We can insert records into the table, but the primary key value must be unique and not NULL.

Example : Insert valid records

```
INSERT INTO Student (id, name, age, course)
VALUES (1, 'Ali', 20, 'B.Sc'),
(2, 'Sara', 21, 'BCA'),
(3, 'Aman', 20, 'B.Sc');
```

Result :

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 21  | BCA    |
| 3  | Aman | 20  | B.Sc   |

## 9. Adding Primary Key using ALTER TABLE

- We can also add a primary key constraint to an existing table using **ALTER TABLE**.

```
CREATE TABLE Teacher (
id INT,
name VARCHAR(50),
subject VARCHAR(50)
);
```



```
ALTER TABLE Teacher
ADD PRIMARY KEY (id);
```

## 2. Characteristics of Primary Key

- Uniqueness**  
Each value in the primary key column must be unique.
- Non-NULL** Key column cannot contain NULL values.
- Single or Composite**  
It can be a single column or a combination of two or more columns.
- Immutable (Stable)**  
The value in a primary key column should not be frequently changed.
- Only One per Table**  
A table can have only one primary key.
- Automatic Indexing**  
A primary key automatically creates a unique index to improve performance.

## 6. Violation of Primary Key Constraint

- If we try to insert a duplicate primary key value or NULL value, it will give an error.

Example 1 : Duplicate Key (Not Allowed)

```
INSERT INTO Student (id, name, age, course)
VALUES (1, 'Neha', 22, 'BCA');
```

Output :

ERROR: Duplicate entry '1' for key 'PRIMARY' (or similar error message)

Example 2 : NULL Value (Not Allowed)

```
INSERT INTO Student (id, name, age, course)
VALUES (NULL, 'Zara', 21, 'B.Sc');
```

Output :

ERROR: Column 'id' cannot be null (or similar error message)

## 10. Dropping Primary Key Constraint

To remove primary key from a table.

```
ALTER TABLE table_name
DROP PRIMARY KEY;
```

Example :

```
ALTER TABLE Student
DROP PRIMARY KEY;
```

## 3. Syntax

### 1. Single Column Primary Key

```
CREATE TABLE table_name (
column_name datatype PRIMARY KEY,
other columns ...,
);
```

### 2. Composite Primary Key (Multiple Columns)

```
CREATE TABLE table_name (
column1 datatype,
column2 datatype,
...
PRIMARY KEY (column1, column2)
);
```

## 7. Composite Primary Key

- A combination of two or more columns can be used as a primary key.
- It ensures uniqueness on the combination of columns.

Example :

```
CREATE TABLE Enroll (
student_id INT,
course_id INT,
enroll_date DATE,
PRIMARY KEY (student_id, course_id)
);
```

Result : Enroll Table Structure

| Column Name | Data Type | Key         |
|-------------|-----------|-------------|
| student_id  | INT       | PRIMARY KEY |
| course_id   | INT       | PRIMARY KEY |
| enroll_date | DATE      | -           |

## 11. Important Points

- Primary key must be unique.
- It cannot be NULL.
- Only one primary key per table.
- Can be single column or composite.
- Helps in maintaining data integrity.
- Automatically creates a unique index.
- Frequently changing primary key values should be avoided.

## 4. Example : Create Table with Primary Key

Example 1 : Single Column Primary Key

```
CREATE TABLE Student (
id INT PRIMARY KEY,
name VARCHAR(50),
age INT,
course VARCHAR(30)
);
```

Result : Student Table Structure

| Column Name | Data Type   | Key         |
|-------------|-------------|-------------|
| id          | INT         | PRIMARY KEY |
| name        | VARCHAR(50) | -           |
| age         | INT         | -           |
| course      | VARCHAR(30) | -           |

## 8. Example : Insert Data in Composite Key

```
INSERT INTO Enroll (student_id, course_id, enroll_date)
VALUES (1, 101, '2024-01-10'),
(1, 102, '2024-02-15'),
(2, 101, '2024-01-12');
```

Result :

| student_id | course_id | enroll_date |
|------------|-----------|-------------|
| 1          | 101       | 2024-01-10  |
| 1          | 102       | 2024-02-15  |
| 2          | 101       | 2024-01-12  |

## 12. Conclusion

- Primary key constraint uniquely identifies each record in a table.
- It prevents duplicate and NULL values.
- It can be single column or composite key.
- It maintains data integrity and improves query performance using indexing.
- It is an important constraint in relational databases.



# Foreign Key Constraint

## 1. Introduction

- A **Foreign Key** is a constraint that establishes a relationship between two tables.
- It ensures **referential integrity** by restricting the values in a column to only those values that exist in the **primary key** of another table.
- It prevents invalid data from being inserted into the table.
- A Foreign Key can contain **duplicate values** and **NULL values** (unless specified as **NOT NULL**).

## 2. Key Points

- It creates a link between a child table and a parent table.
- The child table contains a foreign key column that references the primary key of the parent table.
- The data **type** of the foreign key must be the same as the referenced **primary key**.
- It maintains referential integrity between the tables.
- A foreign key can have duplicate values.
- A foreign key can have **NULL values** unless specified with **NOT NULL**.

## 3. Syntax

### 1. Creating Foreign Key at the time of table creation

```
CREATE TABLE child_table (
    column1 datatype,
    column2 datatype,
    ...
    FOREIGN KEY (foreign_key_column)
    REFERENCES parent_table (primary_key_column)
);
```

### 2. Adding Foreign Key using ALTER TABLE

```
ALTER TABLE child_table
ADD CONSTRAINT constraint_name
FOREIGN KEY (foreign_key_column)
REFERENCES parent_table (primary_key_column);
```

## 4. Example : Create Tables with Foreign Key

### 1. Parent Table (Department)

```
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

### 2. Child Table (Student) with Foreign Key

```
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id)
    REFERENCES Department(dept_id)
);
```

## 5. Example with Data Insertion

Parent Table : Department

| dept_id | dept_name |
|---------|-----------|
| 1       | B.Sc      |
| 2       | BCA       |
| 3       | B.Com     |

Child Table : Student

| id | name | dept_id |
|----|------|---------|
| 1  | Ali  | 1       |
| 2  | Sara | 2       |
| 3  | Aman | 1       |
| 4  | Neha | 3       |

## 6. Valid Insertion (Allowed)

- We can insert a record in the child table only if the foreign key value exists in the parent table.

### Example :

```
INSERT INTO Student (id, name, dept_id)
VALUES (5, 'Zara', 2);
```

### Result :

Record inserted successfully because dept\_id = 2 exists in Department table.

## 7. Invalid Insertion (Not Allowed)

- If we try to insert a foreign key value that does not exist in the parent table, it will give an error.

### Example :

```
INSERT INTO Student (id, name, dept_id)
VALUES (6, 'Rohit', 5);
```

### Result :

**ERROR:** Cannot insert or update a child row: a foreign key constraint fails (because dept\_id = 5 does not exist in Department table).

## 8. NULL and Duplicate Values

- A foreign key can have **duplicate values**.
- A foreign key can have **NULL values** (unless specified as **NOT NULL**).
- Multiple students can belong to the same department.

### Example :

| id | name | dept_id |
|----|------|---------|
| 1  | Ali  | 1       |
| 2  | Sara | 1       |
| 3  | Aman | NULL    |
| 4  | Neha | 2       |

## 9. Referential Integrity Diagram

Parent Table  
(Department)

| dept_id | dept_name |
|---------|-----------|
| 1       | B.Sc      |
| 2       | BCA       |
| 3       | B.Com     |

Child Table  
(Student)

| id | name | dept_id |
|----|------|---------|
| 1  | Ali  | 1       |
| 2  | Sara | 2       |
| 3  | Aman | 1       |
| 4  | Neha | 3       |

Foreign Key  
references  
Primary Key

## 10. Deleting and Updating Behavior

- If we try to **delete** a record from the parent table that is referenced in the child table, it will give an error.
- We can handle this using **ON DELETE** and **ON UPDATE** options.

### Example :

```
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

## 11. ON DELETE and ON UPDATE Options

| Option      | Description                                                       |
|-------------|-------------------------------------------------------------------|
| CASCADE     | Automatically deletes/updates related records in the child table. |
| SET NULL    | Sets the foreign key value to NULL in the child table.            |
| SET DEFAULT | Sets the foreign key value to its default value.                  |
| RESTRICT    | Does not allow deletion or update if child records exist.         |
| NO ACTION   | Same as RESTRICT (default behavior).                              |

## 12. Important Points

- Foreign key maintains referential integrity.
- It links child table with parent table.
- Data type of foreign key and referenced primary key must be same.
- It can contain duplicate values.
- It can contain NULL values (unless NOT NULL is specified).
- It prevents insertion of invalid data.
- ON DELETE and ON UPDATE options control the behavior when parent data is modified or deleted.



# Operations on Table with Constraints

## 1. Creating Table with Constraints

- We can create a table and define various constraints like **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL**, **CHECK** and **DEFAULT** at the time of table creation.
- Syntax:**

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    ...
    CONSTRAINT constraint_name constraint_type
);
```

**Example:** Create Student and Department tables with constraints

### Department Table

```
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
    NOT NULL UNIQUE
);
```

### Student Table

```
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 18),
    dept_id INT,
    FOREIGN KEY (dept_id)
    REFERENCES Department(dept_id)
);
```

## 2. Inserting Data (Valid)

- We can insert records into a table if all constraint conditions are satisfied.
- Syntax:**

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

**Example:** Insert valid records

Insert into Department table:

```
INSERT INTO Department (dept_id, dept_name)
VALUES (1, 'B.Sc');
INSERT INTO Department (dept_id, dept_name)
VALUES (2, 'BCA');
```

Insert into Student table:

```
INSERT INTO Student (id, name, age, dept_id)
VALUES (1, 'Ali', 20, 1);
INSERT INTO Student (id, name, age, dept_id)
VALUES (2, 'Sara', 22, 2);
```

**Result:**

| Department Table |           | Student Table |      |     |         |
|------------------|-----------|---------------|------|-----|---------|
| dept_id          | dept_name | id            | name | age | dept_id |
| 1                | B.Sc      | 1             | Ali  | 20  | 1       |
| 2                | BCA       | 2             | Sara | 22  | 2       |

## 3. Inserting Data (Violation of Constraints)

- If we try to insert a record that violates any constraint, the database will give an error.

**Example 1:** **PRIMARY KEY** violation (duplicate id)

```
INSERT INTO Student (id, name, age, dept_id)
VALUES (1, 'Aman', 21, 1);
```

**Error:** Duplicate entry '1' for key 'PRIMARY'

**Example 2:** **NOT NULL** violation

```
INSERT INTO Student (id, name, age, dept_id)
VALUES (3, NULL, 20, 1);
```

**Error:** Column 'name' cannot be null

**Example 3:** **CHECK** constraint violation

```
INSERT INTO Student (id, name, age, dept_id)
VALUES (4, 'Neha', 15, 1);
```

**Error:** Check constraint 'age >= 18' is violated

**Example 4:** **FOREIGN KEY** violation

```
INSERT INTO Student (id, name, age, dept_id)
VALUES (5, 'Zara', 20, 10);
```

**Error:** Cannot add or update a child row: a foreign key constraint fails

## 4. Updating Data with Constraints

- We can update existing records, but the new values must satisfy all constraints.
- Syntax:**

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

**Example 1:** Valid Update

```
UPDATE Student SET age = 23 WHERE id = 1;
```

**Result:**

| id | name | age | dept_id |
|----|------|-----|---------|
| 1  | Ali  | 23  | 1       |
| 2  | Sara | 22  | 2       |

**Example 2:** Update violating **CHECK** constraint

```
UPDATE Student SET age = 16 WHERE id = 2;
```

**Error:** Check constraint 'age >= 18' is violated

**Example 3:** Update to invalid **FOREIGN KEY**

```
UPDATE Student SET dept_id = 5 WHERE id = 1;
```

**Error:** Cannot add or update a child row: a foreign key constraint fails

## 5. Deleting Data with Constraints

- We can delete records from a table using **DELETE** statement, but we cannot delete a parent record if it is referenced by a child table (foreign key).
- Syntax:**

```
DELETE FROM table_name
WHERE condition;
```

**Example 1:** Delete a student record

```
DELETE FROM Student WHERE id = 2;
```

**Result:**

| id | name | age | dept_id |
|----|------|-----|---------|
| 1  | Ali  | 23  | 1       |

**Example 2:** Delete a department with child records

```
DELETE FROM Department WHERE dept_id = 1;
```

**Error:** Cannot delete or update a parent row: a foreign key constraint fails

## 6. Altering Table to Add Constraints

- We can add constraints to an existing table using **ALTER TABLE**.
- Syntax:**

```
ALTER TABLE table_name
ADD constraint_type (column_name ...);
```

**Example:** Add **UNIQUE** and **CHECK** constraint

```
ALTER TABLE Student
ADD UNIQUE (name);
ALTER TABLE Student
ADD CHECK (age >= 18);
```

## 7. Removing Constraints

- We can remove constraints using **ALTER TABLE**.
- Syntax:**

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

**Example:** Remove **CHECK** constraint

```
ALTER TABLE Student
DROP CONSTRAINT age_check;
```

## 8. Modifying Table Structure with Constraints

- We can modify columns and add constraints while altering table structure.

**Example:** Add a new column with **DEFAULT**

```
ALTER TABLE Student
ADD email VARCHAR(50) DEFAULT 'NA';
```

**Example:** Modify column with **NOT NULL**

```
ALTER TABLE Student
MODIFY name VARCHAR(50) NOT NULL;
```

**Example:** Add **FOREIGN KEY** using **ALTER**

```
ALTER TABLE Student
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_id)
REFERENCES Department(dept_id);
```

## 9. Useful Queries to View Constraints

- We can check the table structure and constraints using database commands.

**Example:** In MySQL

```
DESCRIBE Student;
Or
SHOW CREATE TABLE Student;
```

## 10. Important Points

- All constraint conditions must be satisfied during **INSERT**, **UPDATE** and **DELETE**.
- PRIMARY KEY** and **UNIQUE** prevent duplicate values.
- NOT NULL** does not allow **NULL** values.
- CHECK** ensures values satisfy a condition.
- FOREIGN KEY** maintains referential integrity.
- DEFAULT** provides a default value.
- Constraints help maintain data integrity, consistency and reliability in the database.



# Views : Need , Advantages and Types

## 1. What is a View ?

- A View is a virtual table based on the result set of a SELECT query.
- It does not store data physically like a table, it only stores the query definition.
- It shows data from one or more tables.
- Views are useful for simplifying complex queries and providing security.

## 5. Syntax of Creating a View

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

**Example :** Create a view for student details

```
CREATE VIEW StudentView AS
SELECT id, name, course
FROM Student;
```

## 2. Need of View

- To simplify complex queries.
- To provide security by restricting access to specific columns or rows.
- To present a customized view of data to different users.
- To hide complex SQL logic from end users.
- To maintain data consistency across multiple queries.

## 6. Base Table and View

Base Table : Student

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 21  | BCA    |
| 3  | Aman | 20  | B.Sc   |
| 4  | Neha | 22  | BCA    |

View : StudentView  
(only selected columns)

| id | name | course |
|----|------|--------|
| 1  | Ali  | B.Sc   |
| 2  | Sara | BCA    |
| 3  | Aman | B.Sc   |
| 4  | Neha | BCA    |

## 3. Advantages of Views

- 1 **Security** : Restrict access to specific data (rows/columns).
- 2 **Simplifies Complex Queries** : Makes queries easier to use.
- 3 **Data Independence** : Hides underlying table structure.
- 4 **Reusability** : Same view can be used multiple times.
- 5 **Consistency** : Provides a uniform way to access data.
- 6 **Customization** : Different users can have different views of the same data.

## 4. Disadvantages / Limitations

- Views do not store data (they are virtual).
- Can be slower than direct table access in some cases.
- Not all views are updatable.
- Complex views (with joins, aggregates, etc.) cannot always be used for INSERT, UPDATE or DELETE.

## 7. Using a View

- We can query a view like a table.

**Syntax :**

```
SELECT * FROM view_name;
```

**Example :**

```
SELECT * FROM StudentView;
```

**Result :**

| id | name | course |
|----|------|--------|
| 1  | Ali  | B.Sc   |
| 2  | Sara | BCA    |
| 3  | Aman | B.Sc   |
| 4  | Neha | BCA    |

## 8. Updating a View

- Simple views (based on a single table and without aggregate functions) can be updated.
- Complex views (with joins, GROUP BY, etc.) are read-only.

**Example :** Update a view

```
UPDATE StudentView
SET course = 'B.Sc'
WHERE id = 1;
```

The update will be reflected in the base table if the view is updatable.

**Example :** Insert data using a view

```
INSERT INTO StudentView (id, name, course)
VALUES (5, 'Zara', 'BCA');
```

## 9. Types of Views

Views can be classified mainly into the following types :

- 1 Simple View
- 2 Complex View
- 3 Updatable View
- 4 Non-Updatable View
- 5 Materialized View (Database specific)

## 10. Simple View

- Based on a single table.
- Does not use aggregate functions or GROUP BY.
- Can be updated (INSERT, UPDATE, DELETE).

**Example :**

```
CREATE VIEW SimpleView AS
SELECT id, name
FROM Student;
```

## 11. Complex View

- Based on multiple tables.
- Uses joins, aggregate functions, GROUP BY, etc.
- Generally read-only (not updatable).

**Example :**

```
CREATE VIEW ComplexView AS
SELECT s.id, s.name, d.dept_name
FROM Student s
JOIN Department d
ON s.dept_id = d.dept_id;
```

**Example (Oracle) :**

```
CREATE MATERIALIZED VIEW StudentCount AS
SELECT course, COUNT(*) AS total
FROM Student
GROUP BY course;
```

## 12. Updatable View

- Can be used to perform INSERT, UPDATE and DELETE operations.
- Usually based on a single table and without aggregate functions.

**Example :**

```
CREATE VIEW UpdatableView AS
SELECT id, name, age
FROM Student;

UPDATE UpdatableView
SET age = 21
WHERE id = 2;
```

## 13. Non-Updatable View

- Cannot be used to insert, update or delete data.
- Usually created using joins, GROUP BY, aggregate functions, etc.

**Example :**

```
CREATE VIEW NonUpdatableView AS
SELECT course, COUNT(*) AS total
FROM Student
GROUP BY course;
```

## 14. Materialized View (Database Specific)

- Stores the result set physically on disk (unlike normal view).
- Used to improve performance for complex queries (especially large data).
- Supported in some databases like Oracle.

## 15. Important Points

- 1 Views are virtual tables.
- 2 They do not store data (except materialized views).
- 3 Provide security, simplicity and data consistency.
- 4 Simple views can be updated, complex views are usually read-only.
- 5 Useful in large databases and multi-user environments.



# Index in DBMS (RDBMS) : Need, Advantages and Types

## 1. Introduction to Index

- An Index is a database object that improves the speed of data retrieval operations on a table.
- It works like an index in a book, which helps us find a page quickly.
- Index stores the values of one or more columns in a sorted order along with pointers to the actual rows in the table.
- It is created on a table to speed up SELECT queries, especially those with WHERE, JOIN and ORDER BY clauses.

## 5. How Index Works ?

- When an index is created on a column, the DBMS maintains a separate index structure which stores the column values in a sorted order along with a pointer to the corresponding row.
- When a query is executed with a condition on the indexed column, the DBMS uses the index to quickly locate the rows instead of scanning the entire table.

## 2. Need of Index

- To improve the performance of data retrieval (faster search).
- To reduce the time required to find records in large tables.
- To avoid full table scan.
- To quickly locate rows based on column values.
- Useful in queries with WHERE, JOIN, ORDER BY and GROUP BY.
- Essential in large databases where tables contain millions of records.

## 6. Example to Understand Index

Student Table (without index)

| id | name | age | course |
|----|------|-----|--------|
| 1  | Ali  | 20  | B.Sc   |
| 2  | Sara | 22  | BCA    |
| 3  | Aman | 19  | B.Sc   |
| 4  | Neha | 21  | BCA    |
| 5  | Zara | 20  | B.Sc   |

Index on age column (sorted order)

| age | pointer (row id) |
|-----|------------------|
| 19  | 3                |
| 20  | 1,5              |
| 21  | 4                |
| 22  | 2                |

## 3. Advantages of Index

- 1** **Faster Data Retrieval** : Greatly improves the speed of SELECT queries.
- 2** **Reduces Disk I/O** : Minimizes the number of disk reads.
- 3** **Efficient Searching** : Helps in quickly locating rows.
- 4** **Improves Performance** : Especially in large tables.
- 5** **Useful in Sorting** : Helps in ORDER BY operations.
- 6** **Speeds up JOIN and GROUP BY** : Improves performance of these queries.
- 7** **Maintains Uniqueness** : In case of UNIQUE index, ensures all values are unique.

## 4. Disadvantages of Index

- Extra Storage Space** : Indexes require additional disk space.
- Slower Data Modification** : INSERT, UPDATE and DELETE operations become slower because the index also needs to be updated.
- Maintenance Overhead** : Indexes need to be maintained by the DBMS.
- Not Useful for Small Tables** : For small tables, index may not provide significant performance improvement.
- Too Many Indexes** : Creating too many indexes can degrade performance.

## 7. Syntax of Creating Index

### 1. Create a normal index

```
CREATE INDEX index_name  
ON table_name (column_name);
```

#### Example :

```
CREATE INDEX idx_age  
ON Student (age);
```

### 2. Create a unique index

```
CREATE UNIQUE INDEX index_name  
ON table_name (column_name);
```

#### Example :

```
CREATE UNIQUE INDEX idx_email  
ON Student (email);
```

## 8. Viewing and Dropping Index

### 1. View indexes (syntax may vary by DBMS)

```
-- In MySQL  
SHOW INDEX FROM table_name;  
-- In Oracle  
SELECT * FROM user_indexes  
WHERE table_name = 'STUDENT';
```

### 2. Drop an index

```
DROP INDEX index_name  
ON table_name;
```

#### Example :

```
DROP INDEX idx_age ON Student;
```

## 9. Types of Index

Indexes can be classified into the following types :

- 1** **Primary Index**
- 2** **Unique Index**
- 3** **Secondary Index**
- 4** **Clustered Index**
- 5** **Non-Clustered Index**
- 6** **Composite Index**
- 7** **Function-Based Index (or Expression Index)**

## 10. Primary Index

- Created automatically on the Primary Key column.
- Ensures uniqueness and faster search.
- Only one primary index per table.

#### Example :

```
CREATE TABLE Student (  
id INT PRIMARY KEY,  
name VARCHAR(50)  
);
```

A primary index is automatically created on 'id' column.

## 11. Unique Index

- Ensures that all values in the indexed column are unique.
- It can be created on columns with UNIQUE constraint.
- Multiple unique indexes can be created on a table.

#### Example :

```
CREATE UNIQUE INDEX idx_email  
ON Student(email);
```

## 12. Secondary Index

- Created on a non-primary key column.
- Helps in faster retrieval of records based on non-key columns.
- Multiple secondary indexes can exist on a table.

#### Example :

```
CREATE INDEX idx_name  
ON Student(name);
```

## 17. Important Points

- 1** Indexes improve performance but consume extra storage space.
- 2** Avoid creating too many indexes.
- 3** Choose columns frequently used in WHERE, JOIN or ORDER BY clauses.
- 4** Primary key index is created automatically.
- 5** Indexes need maintenance during INSERT, UPDATE and DELETE.
- 6** Use appropriate type of index based on the requirement.

#### Example :

```
CREATE INDEX idx_upper_name  
ON Student(UPPER(name));
```

## 13. Clustered Index

- Stores the actual data rows in the same order as the index.
- Only one clustered index per table.
- Usually created on the primary key.

## 14. Non-Clustered Index

- Stores index entries separately from the actual data rows.
- Multiple non-clustered indexes can exist.
- Contains a pointer to the actual data.

## 15. Composite Index

- Created on two or more columns.
- Useful for queries with multiple conditions.

#### Example :

```
CREATE INDEX idx_name_age  
ON Student(name, age);
```

## 16. Function-Based Index

- Created on an expression or function applied to a column.
- Helps in queries using functions.



# Joins and Types in SQL

## 1. Introduction to Joins

- A **Join** is used to combine rows from two or more tables based on a related column between them.
- It is used to retrieve data from multiple tables in a single query.
- Joins are mainly used when the tables have a common column (key).
- Joins help to avoid data redundancy and maintain data integrity.
- Examples : Employee-Department, Student-Course etc.

## 2. Types of Joins

- Joins are mainly classified into the following types :

- 1 Inner Join
- 2 Left Join (Left Outer Join)
- 3 Right Join (Right Outer Join)
- 4 Full Join (Full Outer Join)
- 5 Cross Join (Cartesian Join)
- 6 Self Join
- 7 Natural Join
- 8 Equi Join and Non-Equi Join

## 3. Sample Tables Used in Examples

| Employee Table (Emp) |      |        |        |
|----------------------|------|--------|--------|
| EmpID                | Name | DeptID | Salary |
| 1                    | Ali  | 10     | 30000  |
| 2                    | Sara | 20     | 40000  |
| 3                    | Aman | 10     | 35000  |
| 4                    | Neha | 30     | 45000  |
| 5                    | Zara | NULL   | 28000  |

| Department Table (Dept) |          |          |
|-------------------------|----------|----------|
| DeptID                  | DeptName | Location |
| 10                      | CSE      | Pune     |
| 20                      | IT       | Mumbai   |
| 30                      | ECE      | Nagpur   |
| 40                      | ME       | Delhi    |

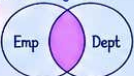
## 4. Important Points

- Joins are used to combine data from multiple tables.
- A join requires at least two tables.
- Joins can be based on equality or a condition.
- Inner join returns only matching rows.
- Outer joins return matching and non-matching rows depending on the type.
- Cross join returns the Cartesian product of both tables.
- Self join is used to join a table with itself.

## 5. Inner Join

- Returns only the matching rows from both tables.
- It excludes non-matching rows.

Venn Diagram :



Syntax :

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Example :

```
SELECT Emp.Name, Dept.DeptName
FROM Emp
INNER JOIN Dept
ON Emp.DeptID = Dept.DeptID;
```

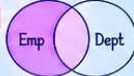
Result :

| Name | DeptName |
|------|----------|
| Ali  | CSE      |
| Sara | IT       |
| Aman | CSE      |
| Neha | ECE      |

## 6. Left Join (Left Outer Join)

- Returns all rows from the left table and matching rows from the right table. If there is no match, **NULL** is returned for the right table columns.

Venn Diagram :



Syntax :

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Example :

```
SELECT Emp.Name, Dept.DeptName
FROM Emp
LEFT JOIN Dept
ON Emp.DeptID = Dept.DeptID;
```

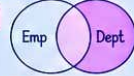
Result :

| Name | DeptName |
|------|----------|
| Ali  | CSE      |
| Sara | IT       |
| Aman | CSE      |
| Neha | ECE      |
| Zara | NULL     |

## 7. Right Join (Right Outer Join)

- Returns all rows from the right table and matching rows from the left table. If there is no match, **NULL** is returned for the left table columns.

Venn Diagram :



Syntax :

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Example :

```
SELECT Emp.Name, Dept.DeptName
FROM Emp
RIGHT JOIN Dept
ON Emp.DeptID = Dept.DeptID;
```

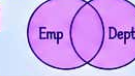
Result :

| Name | DeptName |
|------|----------|
| Ali  | CSE      |
| Sara | IT       |
| Aman | CSE      |
| Neha | ECE      |
| NULL | ME       |

## 8. Full Join (Full Outer Join)

- Returns all rows when there is a match in either left or right table. If there is no match, **NULL** is returned for the missing side.

Venn Diagram :



Syntax :

```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON table1.column = table2.column;
```

Example :

```
SELECT Emp.Name, Dept.DeptName
FROM Emp
FULL OUTER JOIN Dept
ON Emp.DeptID = Dept.DeptID;
```

Result :

| Name | DeptName |
|------|----------|
| Ali  | CSE      |
| Sara | IT       |
| Aman | CSE      |
| Neha | ECE      |
| Zara | NULL     |
| NULL | ME       |

## 9. Cross Join (Cartesian Join)

- Returns the Cartesian product (all possible combinations) of rows from both tables.
- It does not require a join condition.

Venn Diagram :



Syntax :

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

Example :

```
SELECT Emp.Name, Dept.DeptName
FROM Emp
CROSS JOIN Dept;
```

Result : (First Few Rows)

| Name | DeptName |
|------|----------|
| Ali  | CSE      |
| Ali  | IT       |
| Ali  | ECE      |
| Ali  | ME       |
| Sara | CSE      |
| ...  | ...      |

## 10. Self Join

- A table is joined with itself.
- Useful when a table has a hierarchical relationship, such as an employee and their manager.

Syntax :

```
SELECT a.column, b.column
FROM table a
JOIN table b
ON a.common-column = b.common-column;
```

Example : (Employee and Manager)

```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Emp e1
LEFT JOIN Emp e2
ON e1.ManagerID = e2.EmpID;
```

## 11. Natural Join

- Automatically joins tables based on columns with the same name and compatible data types.
- It eliminates duplicate columns in the result.

Syntax :

```
SELECT columns
FROM table1
NATURAL JOIN table2;
```

Example :

```
SELECT EmpID, Name, DeptName
FROM Emp
NATURAL JOIN Dept;
```

## 12. Equi Join

- A join condition uses the equality (=) operator.
- It is the most common type of join.

Example :

```
SELECT Emp.Name, Dept.DeptName
FROM Emp
JOIN Dept
ON Emp.DeptID = Dept.DeptID;
```

## 13. Non-Equi Join

- A join condition uses operators other than equality, such as >, <, >=, <=, BETWEEN.
- It is used when the relationship is based on a range.

Example :

```
SELECT e.Name, s.Grade
FROM Emp e
JOIN SalaryGrade s
ON e.Salary BETWEEN s.MinSal
AND s.MaxSal;
```

## 14. Summary of Join Types

| Join Type     | Description                                                |
|---------------|------------------------------------------------------------|
| Inner Join    | Returns only matching rows.                                |
| Left Join     | All rows from left table + matching rows from right table. |
| Right Join    | All rows from right table + matching rows from left table. |
| Full Join     | All rows from both tables.                                 |
| Cross Join    | Cartesian product of both tables.                          |
| Self Join     | Joins a table with itself.                                 |
| Natural Join  | Joins based on same column names.                          |
| Equi Join     | Uses equality (=) condition.                               |
| Non-Equi Join | Uses non-equality (>, <, >=, <=, BETWEEN).                 |



# Union and Types in SQL

## 1. Introduction to UNION

- UNION is a set operator in SQL.
- It is used to combine the result sets of two or more SELECT statements.
- It returns only distinct (unique) rows by default (duplicates are removed).
- The number of columns and their data types must be the same in all SELECT statements.
- The column names in the result are taken from the first SELECT statement.

## 2. Rules for Using UNION

- Both SELECT statements must have the same number of columns.
- The data type of corresponding columns must be compatible.
- The order of columns should be the same.
- Column names in the result will be taken from the first SELECT statement. We can use UNION to combine more than two SELECT statements.
- ORDER BY (if used) should be at the end of the last SELECT statement.

## 3. Syntax of UNION

```
SELECT column1, column2, ...  
FROM table1  
UNION  
SELECT column1, column2, ...  
FROM table2  
[ORDER BY column_name];
```

## 4. Example of UNION

Table : Student1

| id | name | course |
|----|------|--------|
| 1  | Ali  | B.Sc   |
| 2  | Sara | BCA    |
| 3  | Aman | B.Com  |

Table : Student2

| id | name | course |
|----|------|--------|
| 3  | Aman | B.Com  |
| 4  | Neha | B.Sc   |
| 5  | Zara | BCA    |

Result : (Duplicates Removed)

| id | name | course |
|----|------|--------|
| 1  | Ali  | B.Sc   |
| 2  | Sara | BCA    |
| 3  | Aman | B.Com  |
| 4  | Neha | B.Sc   |
| 5  | Zara | BCA    |

Query :

```
SELECT id, name, course FROM Student1  
UNION  
SELECT id, name, course FROM Student2;
```

## 5. Types of UNION

- UNION can be mainly classified into two types:

1 UNION (Default)

2 UNION ALL

## 6. UNION (Default)

- Returns only distinct (unique) rows.
- Duplicate rows are automatically removed.
- It is the commonly used type.

Example :

```
SELECT id, name FROM A  
UNION  
SELECT id, name FROM B;
```

Table : A

| id | name |
|----|------|
| 1  | Ali  |
| 2  | Sara |
| 3  | Aman |

Table : B

| id | name |
|----|------|
| 3  | Aman |
| 4  | Neha |
| 5  | Zara |

Result : (Duplicates Removed)

| id | name |
|----|------|
| 1  | Ali  |
| 2  | Sara |
| 3  | Aman |
| 4  | Neha |
| 5  | Zara |

## 7. UNION ALL

- Returns all rows including duplicate rows.
- It does not remove duplicates.
- It is faster than UNION because it does not perform duplicate elimination.

Example :

```
SELECT id, name FROM A  
UNION ALL  
SELECT id, name FROM B;
```

Table : A

| id | name |
|----|------|
| 1  | Ali  |
| 2  | Sara |
| 3  | Aman |

Table : B

| id | name |
|----|------|
| 3  | Aman |
| 4  | Neha |
| 5  | Zara |

Result : (Duplicates Included)

| id | name |
|----|------|
| 1  | Ali  |
| 2  | Sara |
| 3  | Aman |
| 3  | Aman |
| 4  | Neha |
| 5  | Zara |

## 8. Using UNION with Multiple Tables

- UNION can be used to combine more than two SELECT statements.

Example :

```
SELECT name FROM Student  
UNION  
SELECT name FROM Teacher  
UNION  
SELECT name FROM Staff  
ORDER BY name;
```

## 9. Differences between UNION and UNION ALL

| Point         | UNION                              | UNION ALL                                  |
|---------------|------------------------------------|--------------------------------------------|
| Duplicates    | Removed (only unique rows)         | Included (all rows)                        |
| Performance   | Slower (checks for duplicates)     | Faster (no duplicate check)                |
| Use case      | When we need only unique data      | When we need all data including duplicates |
| Sorting       | Automatically sorts (in some DBMS) | Does not sort                              |
| Commonly used | When distinct values are required  | When performance is priority               |

## 10. Important Points

- UNION and UNION ALL both require the same number of columns and compatible data types.
- UNION removes duplicates, UNION ALL keeps duplicates.
- Column names in the result are taken from the first SELECT.
- ORDER BY can be used only once at the end.
- UNION can combine more than two SELECT statements.
- UNION is useful to combine similar data from multiple tables, but it cannot be used to combine different structures.
- UNION is mainly used for report generation and data integration.



# In-built Functions in SQL

## 1. Character Functions (String Functions)

- Character functions are used to perform operations on character or string data.
- These functions can be used with data type **CHAR**, **VARCHAR**, etc.

| No. | Function  | Description                                      | Syntax                                  | Example                             | Result           |
|-----|-----------|--------------------------------------------------|-----------------------------------------|-------------------------------------|------------------|
| 1   | UPPER()   | Converts a string to uppercase.                  | UPPER(string)                           | SELECT UPPER('forsko');             | FORSKO           |
| 2   | LOWER()   | Converts a string to lowercase.                  | LOWER(string)                           | SELECT LOWER('FORSKO');             | forsko           |
| 3   | INITCAP() | Converts first letter of each word to uppercase. | INITCAP(string)                         | SELECT INITCAP('computer science'); | Computer Science |
| 4   | LENGTH()  | Returns the number of characters.                | LENGTH(string)                          | SELECT LENGTH('FORSKO');            | 6                |
| 5   | SUBSTR()  | Extracts a substring from a string.              | SUBSTR(string, start, length)           | SELECT SUBSTR('FORSKO', 2, 4);      | ORSK             |
| 6   | INSTR()   | Returns the position of a substring.             | INSTR(string, substring)                | SELECT INSTR('FORSKO', 'S');        | 4                |
| 7   | CONCAT()  | Joins two strings.                               | CONCAT(string1, string2)                | SELECT CONCAT('Data', 'Base');      | DataBase         |
| 8   | TRIM()    | Removes leading and trailing spaces.             | TRIM(string)                            | SELECT TRIM(' FORSKO ');            | FORSKO           |
| 9   | LTRIM()   | Removes leading spaces.                          | LTRIM(string)                           | SELECT LTRIM(' FORSKO');            | FORSKO           |
| 10  | RTRIM()   | Removes trailing spaces.                         | RTRIM(string)                           | SELECT RTRIM('FORSKO ');            | FORSKO           |
| 11  | REPLACE() | Replaces a substring with another string.        | REPLACE(string, old_string, new_string) | SELECT REPLACE('FORSKO', 'O', '0'); | FORSKO           |
| 12  | LPAD()    | Left pads a string to specified length.          | LPAD(string, length, pad_char)          | SELECT LPAD('CS', 5, '*');          | ***CS            |
| 13  | RPAD()    | Right pads a string to specified length.         | RPAD(string, length, pad_char)          | SELECT RPAD('CS', 5, '*');          | CS***            |
| 14  | ASCII()   | Returns ASCII value of first character.          | ASCII(string)                           | SELECT ASCII('A');                  | 65               |
| 15  | CHR()     | Returns character for ASCII value.               | CHR(number)                             | SELECT CHR(65);                     | A                |

### Example with Table

Table : Student

| id | name | email            |
|----|------|------------------|
| 1  | Ali  | ali123@gmail.com |
| 2  | Sara | sara@gmail.com   |
| 3  | Aman | aman@yahoo.com   |
| 4  | Neha | neha@rediff.com  |

### Some Queries using Character Functions

```

SELECT UPPER(name) FROM Student;
SELECT LOWER(email) FROM Student;
SELECT LENGTH(name) FROM Student;
SELECT SUBSTR(name, 1, 3) FROM Student;
SELECT INSTR(email, '@') FROM Student;
SELECT REPLACE(name, 'a', 'A') FROM Student;

```

## 2. Number Functions

- Number functions are used to perform mathematical operations on numeric data.
- These functions can be used with data type **NUMBER**, **INT**, **FLOAT** etc.

| No. | Function           | Description                                                       | Syntax                        | Example                     | Result          |
|-----|--------------------|-------------------------------------------------------------------|-------------------------------|-----------------------------|-----------------|
| 1   | ROUND()            | Rounds a number to specified decimal places.                      | ROUND(number, decimal_places) | SELECT ROUND(123.456, 2);   | 123.46          |
| 2   | TRUNC()            | Truncates a number to specified decimal places.                   | TRUNC(number, decimal_places) | SELECT TRUNC(123.456, 2);   | 123.45          |
| 3   | CEIL() / CEILING() | Returns the smallest integer greater than or equal to the number. | CEIL(number)                  | SELECT CEIL(12.3);          | 13              |
| 4   | FLOOR()            | Returns the largest integer less than or equal to the number.     | FLOOR(number)                 | SELECT FLOOR(12.7);         | 12              |
| 5   | ABS()              | Returns the absolute (positive) value.                            | ABS(number)                   | SELECT ABS(-25);            | 25              |
| 6   | MOD()              | Returns the remainder after division.                             | MOD(number1, number2)         | SELECT MOD(10, 3);          | 1               |
| 7   | POWER()            | Returns base raised to the power.                                 | POWER(base, exp)              | SELECT POWER(2, 3);         | 8               |
| 8   | SQRT()             | Returns the square root of a number.                              | SQRT(number)                  | SELECT SQRT(49);            | 7               |
| 9   | EXP()              | Returns e raised to the power of the number.                      | EXP(number)                   | SELECT EXP(1);              | 2.71828         |
| 10  | LOG()              | Returns natural logarithm (base e).                               | LOG(number)                   | SELECT LOG(10);             | 2.30259         |
| 11  | LOG10()            | Returns logarithm base 10.                                        | LOG10(number)                 | SELECT LOG10(100);          | 2               |
| 12  | GREATEST()         | Returns the largest value from a list.                            | GREATEST(n1, n2, ...)         | SELECT GREATEST(10, 25, 5); | 25              |
| 13  | LEAST()            | Returns the smallest value from a list.                           | LEAST(n1, n2, ...)            | SELECT LEAST(10, 25, 5);    | 5               |
| 14  | SIGN()             | Returns -1, 0 or 1 (negative, zero or positive).                  | SIGN(number)                  | SELECT SIGN(-20);           | -1              |
| 15  | RAND()             | Returns a random number between 0 and 1 (DBMS specific).          | RAND()                        | SELECT RAND();              | 0.6843 (varies) |

### Example with Table

Table : Marks

| id | name | marks  |
|----|------|--------|
| 1  | Ali  | 78.456 |
| 2  | Sara | 56.789 |
| 3  | Aman | 92.130 |
| 4  | Neha | 41.987 |

### Some Queries using Number Functions

```

SELECT ROUND(marks, 1) FROM Marks;
SELECT TRUNC(marks, 0) FROM Marks;
SELECT CEIL(marks) FROM Marks;
SELECT FLOOR(marks) FROM Marks;
SELECT ABS(-marks) FROM Marks;
SELECT POWER(marks, 2) FROM Marks;
SELECT SQRT(marks) FROM Marks;

```

## 3. Important Points

- Character functions are used with string data types (CHAR, VARCHAR etc.).
- Number functions are used with numeric data types (NUMBER, INT, FLOAT etc.).
- Functions can be used in **SELECT**, **WHERE**, **ORDER BY** and other clauses.
- Some functions (like **RAND()**) may vary depending on the **DBMS** (MySQL, Oracle, etc.).

- Functions can be nested. (Example: **UPPER(SUBSTR(name, 1, 3))**)
- Functions can be used with columns, literals, expressions or another function.
- Using functions in **WHERE** clause may affect performance.
- Always use appropriate function as per requirement and data type.



# In-built Functions in SQL : Date Functions and Group Functions

## 1. Introduction to Date Functions

- Date functions are used to perform operations on **DATE** and **TIME** data.
- They help to extract, manipulate and format date and time values.
- These functions are useful in retrieving specific parts (year, month, day), performing calculations and comparing dates.
- The data type used is **DATE** or **TIMESTAMP** (depending on the DBMS).
- Syntax and function names may vary slightly in different databases (MySQL, Oracle, SQL Server).

## 2. Commonly Used Date Functions

- **CURRENT\_DATE / SYSDATE** : Returns the current date.
- **CURRENT\_TIME / SYSTIME** : Returns the current time.
- **NOW()** : Returns current date and time (MySQL).
- **EXTRACT()** : Extracts a specific part of a date.
- **DATEADD()** : Adds a specified interval to a date (SQL Server).
- **DATEDIFF()** : Returns the difference between two dates.
- **TO\_CHAR() / DATE\_FORMAT()** : Converts a date to a specified format.

## 3. List of Date Functions

| No. | Function                | Description                                       | Syntax                                 | Example                                                   | Result (Assuming current date is 2026-10-03) |
|-----|-------------------------|---------------------------------------------------|----------------------------------------|-----------------------------------------------------------|----------------------------------------------|
| 1   | <b>CURRENT_DATE</b>     | Returns current date.                             | <b>CURRENT_DATE</b>                    | <b>SELECT CURRENT_DATE;</b>                               | 2026-10-03                                   |
| 2   | <b>CURRENT_TIME</b>     | Returns current time.                             | <b>CURRENT_TIME</b>                    | <b>SELECT CURRENT_TIME;</b>                               | 16:25:30                                     |
| 3   | <b>NOW()</b>            | Returns current date and time.                    | <b>NOW()</b>                           | <b>SELECT NOW();</b>                                      | 2026-10-03 16:25:30                          |
| 4   | <b>EXTRACT()</b>        | Extracts part of a date (year, month, day).       | <b>EXTRACT(part FROM date)</b>         | <b>SELECT EXTRACT(YEAR FROM '2026-10-03');</b>            | 2026                                         |
| 5   | <b>DATEADD()</b>        | Adds a specified interval to a date (SQL Server). | <b>DATEADD(interval, number, date)</b> | <b>SELECT DATEADD(DAY, 5, '2026-10-03');</b>              | 2026-10-08                                   |
| 6   | <b>DATEDIFF()</b>       | Returns the difference between two dates.         | <b>DATEDIFF(date1, date2)</b>          | <b>SELECT DATEDIFF('2026-10-03', '2026-09-25');</b>       | 8 (days)                                     |
| 7   | <b>TO_CHAR()</b>        | Converts a date to specified format (Oracle).     | <b>TO_CHAR(date, 'format')</b>         | <b>SELECT TO_CHAR('2026-10-03', 'DD-MON-YYYY');</b>       | 03-OCT-2026                                  |
| 8   | <b>DATE_FORMAT()</b>    | Formats a date (MySQL).                           | <b>DATE_FORMAT(date, format)</b>       | <b>SELECT DATE_FORMAT('2026-10-03', '%d-%m-%Y');</b>      | 03-10-2026                                   |
| 9   | <b>ADD_MONTHS()</b>     | Adds months to a date (Oracle).                   | <b>ADD_MONTHS(date, number)</b>        | <b>SELECT ADD_MONTHS('2026-10-03', 3);</b>                | 2027-01-03                                   |
| 10  | <b>LAST_DAY()</b>       | Returns the last day of the month.                | <b>LAST_DAY(date)</b>                  | <b>SELECT LAST_DAY('2026-02-10');</b>                     | 2026-02-28                                   |
| 11  | <b>MONTHS_BETWEEN()</b> | Returns difference in months (Oracle).            | <b>MONTHS_BETWEEN(date1, date2)</b>    | <b>SELECT MONTHS_BETWEEN('2026-10-03', '2026-07-03');</b> | 3                                            |
| 12  | <b>NEXT_DAY()</b>       | Returns the next specified weekday.               | <b>NEXT_DAY(date, 'day')</b>           | <b>SELECT NEXT_DAY('2026-10-03', 'MONDAY');</b>           | 2026-10-05                                   |
| 13  | <b>TRUNC()</b>          | Truncates time part from date.                    | <b>TRUNC(date)</b>                     | <b>SELECT TRUNC('2026-10-03 16:25:30');</b>               | 2026-10-03                                   |

## 4. Examples with Table

Table : Employee

| id | name | join_date  | salary |
|----|------|------------|--------|
| 1  | Ali  | 2024-01-15 | 30000  |
| 2  | Sara | 2023-06-10 | 40000  |
| 3  | Aman | 2025-02-20 | 35000  |
| 4  | Neha | 2026-09-05 | 45000  |

### Some Queries

```

SELECT name, EXTRACT(YEAR FROM join_date) FROM Employee; -- Returns year of joining
SELECT name, DATEDIFF(CURRENT_DATE, join_date) FROM Employee; -- Days since joining
SELECT name, DATE_FORMAT(join_date, '%d-%m-%Y') FROM Employee; -- Formatted date
SELECT name, ADD_MONTHS(join_date, 6) FROM Employee; -- 6 months after joining
SELECT name, LAST_DAY(join_date) FROM Employee; -- Last day of join month
    
```

## 1. Introduction to Group Functions

- Group functions (also called Aggregate functions) are used to perform calculations on a set of rows and return a single result.
- They operate on multiple rows from a table.
- These functions are commonly used with the **GROUP BY** clause.
- They ignore **NULL** values (except **COUNT(\*)**, which counts all rows).
- Group functions are useful for summarizing data such as totals, averages, minimum, maximum, etc.

## 2. Commonly Used Group Functions

- COUNT()** : Counts the number of rows.
- SUM()** : Returns the total sum of a numeric column.
- AVG()** : Returns the average value.
- MIN()** : Returns the minimum value.
- MAX()** : Returns the maximum value.

## 3. List of Group Functions

| No. | Function       | Description                                    | Syntax                                  | Example                                  | Result (From given table) |
|-----|----------------|------------------------------------------------|-----------------------------------------|------------------------------------------|---------------------------|
| 1   | <b>COUNT()</b> | Counts the number of rows.                     | <b>COUNT(*)</b> or <b>COUNT(column)</b> | <b>SELECT COUNT(*) FROM Employee;</b>    | 4                         |
| 2   | <b>SUM()</b>   | Returns the total sum of a numeric column.     | <b>SUM(column)</b>                      | <b>SELECT SUM(salary) FROM Employee;</b> | 155000                    |
| 3   | <b>AVG()</b>   | Returns the average value of a numeric column. | <b>AVG(column)</b>                      | <b>SELECT AVG(salary) FROM Employee;</b> | 38750                     |
| 4   | <b>MIN()</b>   | Returns the minimum value.                     | <b>MIN(column)</b>                      | <b>SELECT MIN(salary) FROM Employee;</b> | 30000                     |
| 5   | <b>MAX()</b>   | Returns the maximum value.                     | <b>MAX(column)</b>                      | <b>SELECT MAX(salary) FROM Employee;</b> | 45000                     |

## 4. Example with Table

Table : Employee

| id | name | dept | salary |
|----|------|------|--------|
| 1  | Ali  | CSE  | 30000  |
| 2  | Sara | IT   | 40000  |
| 3  | Aman | CSE  | 35000  |
| 4  | Neha | IT   | 45000  |
| 5  | Zara | ECE  | 38000  |

## 5. Examples of Group Functions

| Query                                    | Result   |
|------------------------------------------|----------|
| <b>SELECT COUNT(*) FROM Employee;</b>    | → 5      |
| <b>SELECT SUM(salary) FROM Employee;</b> | → 186000 |
| <b>SELECT AVG(salary) FROM Employee;</b> | → 37200  |
| <b>SELECT MIN(salary) FROM Employee;</b> | → 30000  |
| <b>SELECT MAX(salary) FROM Employee;</b> | → 45000  |

## 6. Using Group Functions with GROUP BY

- The **GROUP BY** clause is used to group rows.
- that have the same values in specified columns.
- Group functions are applied to each group separately.
- It helps to get summary/aggregate results for each group.

### Example :

```

SELECT dept, COUNT(*), SUM(salary)
FROM Employee
GROUP BY dept;
    
```

### Result :

| dept | COUNT(*) | SUM(salary) |
|------|----------|-------------|
| CSE  | 2        | 65000       |
| IT   | 2        | 85000       |
| ECE  | 1        | 38000       |

## 7. Important Points

- Group functions return a single value.
- They ignore **NULL** values (except **COUNT(\*)**).
- They can be used with **GROUP BY** clause.
- They are mainly used on numeric or date columns.
- They are helpful for data analysis and reporting.



# In-built Functions in SQL : Conversion Functions

## 1. Introduction to Conversion Functions

- Conversion functions are used to convert data from one data type to another data type.
- They are useful when we need to perform operations on data of different types (e.g., convert string to number, number to string, date to string, etc.).
- These functions help in **data formatting**, report generation and **data manipulation**.
- Different DBMS (Oracle, MySQL, SQL Server) provide different conversion functions.

## 5. TO\_NUMBER() Function

- Converts a character string to a number.
- Syntax :** `TO_NUMBER(char, 'format')`

Example :

```
SELECT TO_NUMBER('12,345', '99,999')
FROM dual;
```

Result :

```
TO_NUMBER('12,345', '99,999')
12345
```

## 9. NULLIF() Function

- Returns **NULL** if two expressions are equal, otherwise returns the first expression.
- Syntax :** `NULLIF(expression1, expression2)`

Example :

```
SELECT NULLIF(10, 10) FROM dual;
```

Result :

```
NULLIF(10, 10)
NULL
```

## 2. Commonly Used Conversion Functions

- TO\_CHAR()** : Converts a date/number to a character string.
- TO\_DATE()** : Converts a character string to a date.
- TO\_NUMBER()** : Converts a character string to a number.
- CAST()** : Converts a value from one data type to another (standard SQL).
- CONVERT()** : Converts a value from one data type to another (mainly in SQL Server).
- NVL()** : Replaces **NULL** with a specified value (Oracle).
- NULLIF()** : Returns **NULL** if two expressions are equal, otherwise returns the first expression.
- COALESCE()** : Returns the first non-**NULL** value from a list of expressions (standard SQL).

## 6. CAST() Function (Standard SQL)

- Converts a value from one data type to another data type.
- Syntax :** `CAST(expression AS data_type)`

Example :

```
SELECT CAST(123 AS VARCHAR(10))
FROM dual;
```

Result :

```
CAST(123 AS VARCHAR(10))
123
```

## 10. COALESCE() Function (Standard SQL)

- Returns the first non-**NULL** value from a list of expressions.
- Syntax :** `COALESCE(expr1, expr2, expr3, ...)`

Example :

```
SELECT COALESCE(NULL, NULL, 'FORSKO', 'N/A')
FROM dual;
```

Result :

```
COALESCE(NULL, NULL, 'FORSKO', 'N/A')
FORSKO
```

## 3. TO\_CHAR() Function

- Converts a number or date to a character string in a specified format.

- Syntax :** `TO_CHAR(expression, 'format')`

Example 1 : Number to string

```
SELECT TO_CHAR(12345, '99999')
FROM dual;
```

Result :

```
TO_CHAR(12345, '99999')
12345
```

Example 2 : Date to string

```
SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY')
FROM dual;
```

Result :

```
TO_CHAR(SYSDATE, 'DD-MON-YYYY')
03-OCT-2026
```

## 7. CONVERT() Function (SQL Server)

- Converts a value from one data type to another (data type specified).
- Syntax :** `CONVERT(data_type, expression, style)`

Example :

```
SELECT CONVERT(VARCHAR(10), GETDATE(), 103);
```

Result :

```
CONVERT(VARCHAR(10), GETDATE(), 103)
03/10/2026
```

## 11. Important Points

- Conversion functions are helpful for data type compatibility and formatting.
- TO\_CHAR()** and **TO\_DATE()** are mainly used for date and number formatting (Oracle).
- TO\_NUMBER()** converts string to number.
- CAST()** is a standard SQL function.
- CONVERT()** is mainly used in SQL Server.
- NVL()** and **COALESCE()** are used to handle **NULL** values.
- NULLIF()** helps in conditional **NULL** return.
- These functions are widely used in **SELECT**, **INSERT**, **UPDATE** and in expressions.

## 4. TO\_DATE() Function

- Converts a character string to a date value.
- Syntax :** `TO_DATE(char, 'format')`

Example :

```
SELECT TO_DATE('03-10-2026', 'DD-MM-YYYY')
FROM dual;
```

Result :

```
TO_DATE('03-10-2026', 'DD-MM-YYYY')
03-OCT-2026
```

## 8. NVL() Function (Oracle)

- Replaces **NULL** with a specified value.
- Syntax :** `NVL(expression, default_value)`

Example :

```
SELECT NVL(commission, 0) AS comm
FROM Employee;
```

Result :

| id | name | commission | comm |
|----|------|------------|------|
| 1  | Ali  | 5000       | 5000 |
| 2  | Sara | NULL       | 0    |
| 3  | Aman | 3000       | 3000 |

## 12. Summary Table

| Function           | Purpose                                           |
|--------------------|---------------------------------------------------|
| <b>TO_CHAR()</b>   | Convert number/date to string                     |
| <b>TO_DATE()</b>   | Convert string to date                            |
| <b>TO_NUMBER()</b> | Convert string to number                          |
| <b>CAST()</b>      | Convert value to another data type (standard SQL) |
| <b>CONVERT()</b>   | Convert value to another data type (SQL Server)   |
| <b>NVL()</b>       | Replace <b>NULL</b> with a value (Oracle)         |
| <b>NULLIF()</b>    | Return <b>NULL</b> if two expressions are equal   |
| <b>COALESCE()</b>  | Return first non- <b>NULL</b> value               |



# Use of Alias, NULL (NVL) and Sub Queries in SQL

## 1. Alias in SQL (AS)

- Alias is a temporary name given to a column or table.
- It is used to make the output more readable.
- Alias exists only for the duration of the query.
- It is created using the **AS** keyword (optional in some cases).
- Alias can be used for columns, tables and expressions.

## 2. Types of Alias

- Column Alias** : To give a new name to a column in the result.
- Table Alias** : To give a short name to a table.
- Expression Alias** : To give a name to an expression or calculated column.

## 3. Column Alias

**Syntax :**

```
SELECT column_name AS alias_name  
FROM table_name;
```

**Example 1 :**

```
SELECT name AS Student_Name,  
salary AS Monthly_Salary  
FROM Employee;
```

**Result :**

| Student_Name | Monthly_Salary |
|--------------|----------------|
| Ali          | 30000          |
| Sara         | 40000          |
| Aman         | 35000          |
| Neha         | 45000          |

## 6. Important Points (Alias)

- Alias is optional but recommended for better readability.
- Use **AS** keyword (can be omitted in column alias).
- In table alias, **AS** is usually not used (optional).
- Alias name should be meaningful.
- Alias cannot be used in **WHERE** clause (it can be used in **ORDER BY**).
- It is useful in expressions, joins and when column names are long.
- Alias helps avoid column name ambiguity in joins.

## 4. Table Alias

**Syntax :**

```
SELECT t.column1, t.column2  
FROM table_name AS t;
```

**Example 2 :**

```
SELECT e.name, e.salary  
FROM Employee AS e;
```

## 5. Expression Alias

**Syntax :**

```
SELECT expression AS alias_name  
FROM table_name;
```

**Example 3 :**

```
SELECT name, salary*12 AS  
Annual_Salary  
FROM Employee;
```

**Result :**

| name | Annual_Salary |
|------|---------------|
| Ali  | 360000        |
| Sara | 480000        |
| Aman | 420000        |
| Neha | 540000        |

## 1. NULL in SQL

- NULL** represents a missing, unknown or undefined value.
- It is not zero and not a blank space.
- It means there is no **value** in that column.
- NULL** is used when the data is not available or not applicable.
- Arithmetic operations with **NULL** always return **NULL**.
- To check **NULL** we use **IS NULL** and **IS NOT NULL** (not = NULL).

## 2. Example of NULL

| id | name | dept | salary |
|----|------|------|--------|
| 1  | Ali  | CSE  | 30000  |
| 2  | Sara | IT   | NULL   |
| 3  | Aman | ECE  | 35000  |
| 4  | Neha | ME   | NULL   |

## 3. Checking NULL

**Syntax :**

```
SELECT column_name  
FROM table_name  
WHERE column_name IS NULL;
```

**Example :**

```
SELECT * FROM Employee  
WHERE salary IS NULL;
```

**Result :**

| id | name | dept | salary |
|----|------|------|--------|
| 2  | Sara | IT   | NULL   |
| 4  | Neha | ME   | NULL   |

## 4. NVL Function (Oracle)

- NVL()** is an in-built function in Oracle.
- It replaces **NULL** with a specified value.

**Syntax :** **NVL(expression, replacement\_value)**

**Example 1 :**

```
SELECT name, NVL(salary, 0) AS Salary  
FROM Employee;
```

**Result :**

| name | Salary |
|------|--------|
| Ali  | 30000  |
| Sara | 0      |
| Aman | 35000  |
| Neha | 0      |

**Example 2 :**

```
SELECT name, NVL(dept, 'Not  
Assigned') AS Department  
FROM Employee;
```

**Result :**

| name | Department |
|------|------------|
| Ali  | CSE        |
| Sara | IT         |
| Aman | ECE        |
| Neha | ME         |

## 5. Important Points (NULL and NVL)

- NULL** is not a number and not a blank space.
- Use **IS NULL** / **IS NOT NULL** to check **NULL** values.
- Use **NVL()** to replace **NULL** with a default value.
- NVL()** is commonly used in **SELECT**, **WHERE**, **ORDER BY** and expressions.
- NVL()** is an Oracle specific function (In MySQL we use **IFNULL()**).
- We can replace **NULL** with string, number or any expression.
- Useful for handling missing data and avoiding errors in calculations.

## 1. Introduction to Sub Queries

- A Sub query (or Inner query) is a query inside another query.
- It is used to retrieve data that will be used by the outer query.
- Sub query is written within parentheses (**()**).
- It can return a single value, a single row or multiple rows.
- Sub queries can be used in **SELECT**, **FROM**, **WHERE** and **HAVING** clause.
- Sub query is executed first and its result is used by the outer query.
- It helps in filtering data, performing comparisons and complex queries.

## 2. Types of Sub Queries

- Single Row Sub Query** : Returns only one row (one value).
- Multiple Row Sub Query** : Returns multiple rows (multiple values).
- Multiple Column Sub Query** : Returns multiple columns.
- Correlated Sub Query** : Depends on the outer query (executed for each row).
- Non-Correlated Sub Query** : Executes independently (result does not depend on outer query).

## 3. Single Row Sub Query

**Example :**

Find employees who have the same salary as the employee 'Ali'.

```
SELECT name, salary  
FROM Employee  
WHERE salary = (SELECT salary  
FROM Employee  
WHERE name = 'Ali');
```

## 5. Multiple Column Sub Query

**Example :**

Find employees whose (dept, salary) match with records in another table.

```
SELECT name, dept, salary  
FROM Employee  
WHERE (dept, salary) IN (SELECT dept, salary  
FROM Tempmp);
```

## 7. Sub Query in FROM Clause

**Example :**

Find average salary of each department using sub query in FROM.

```
SELECT dept, avg_sal  
FROM (SELECT dept, AVG(salary) AS avg_sal  
FROM Employee  
GROUP BY dept) d;
```

## 4. Multiple Row Sub Query

**Example :**

Find employees who work in the departments present in the 'Department' table.

```
SELECT name, dept  
FROM Employee  
WHERE dept IN (SELECT DeptName  
FROM Department);
```

## 6. Correlated Sub Query

**Example :**

Find employees whose salary is greater than the average salary of their department.

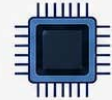
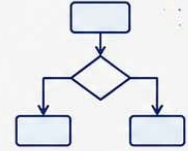
```
SELECT name, dept, salary  
FROM Employee e1  
WHERE salary > (SELECT AVG(salary)  
FROM Employee e2  
WHERE e1.dept = e2.dept);
```

## 8. Important Points (Sub Queries)

- Sub query is executed before the outer query.
- It can return single value, multiple values or multiple columns.
- Use parentheses (**()**) for sub query.
- Sub queries can be used in **SELECT**, **FROM**, **WHERE** and **HAVING**.
- Use **IN**, **ANY**, **ALL** with multiple row sub queries.
- Correlated sub query executes once for each row of the outer query.
- Helps in solving complex queries and filtering data.



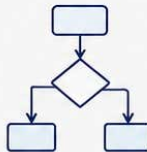
1010101010  
0101010101  
1010101010  
0101010101



# Unit - III



10110  
01001  
10101





# PL/SQL : Introduction and Data Types

## 1. Introduction to PL/SQL

- PL/SQL (Procedural Language/ Structured Query Language) is an extension of SQL developed by Oracle.
- It is a procedural language that combines the data manipulation power of SQL with the procedural features of programming languages.
- PL/SQL is used to write blocks of code (programs) and execute them in the Oracle database server.
- It provides constructs like variables, constants, data types, control statements, loops, cursors, procedures, functions, etc.
- PL/SQL code is executed on the server side, which improves performance, security and reduces network traffic.
- It is widely used for developing database applications, stored procedures, triggers and functions.

## 2. Features of PL/SQL

- Combines SQL with procedural constructs.
- Supports variables, constants and data types.
- Includes control structures (IF, CASE).
- Supports loops (LOOP, FOR, WHILE).
- Allows creation of procedures, functions, packages, triggers.
- Provides exception handling.
- Modular and reusable code.
- Executed on the server side (better performance and security).

## 3. Advantages of PL/SQL

- Improved performance (less network traffic).
- Better security (code stored in database).
- Modular programming (procedures, functions).
- Reusability of code.
- Easy error handling (exception handling).
- Maintainability and readability.
- Suitable for complex business logic.
- Integration with Oracle database features.

## 4. Structure of a PL/SQL Block

Every PL/SQL program is written in blocks. It has three main sections:

```
DECLARE
-- declaration section (optional)
BEGIN
-- executable section (mandatory)
EXCEPTION
-- exception handling section (optional)
END;
```

## 5. Example of a Simple PL/SQL Block

```
DECLARE
v_num NUMBER := 10;
BEGIN
DBMS_OUTPUT.PUT_LINE('Value = ' || v_num);
END;
```

Output :

Value = 10

## 6. PL/SQL Identifiers

- Identifiers are user-defined names used for variables, constants, procedures, functions, etc.
- Rules for identifiers :
  - Must start with a letter.
  - Can contain letters, numbers, \_, \$ and #.
  - Cannot start with a number.
  - Cannot be a reserved keyword.
  - Maximum length is 30 characters (Oracle).
  - Not case-sensitive (but conventionally written in lowercase or mixed case).

## 7. PL/SQL Data Types (Overview)

- PL/SQL provides various data types to store values in variables and constants.
- These data types are similar to SQL data types but with some additional types.
- Data types are mainly classified into the following categories :
  - Scalar Data Types (Single value)
  - Composite Data Types (Collection of values)
  - Reference Data Types (Pointers)
  - LOB Data Types (Large objects)
  - %TYPE and %ROWTYPE (for database compatibility)

## 8. Scalar Data Types

- Scalar data types store a single value.
- These are the most commonly used data types in PL/SQL.

| Data Type | Description                                       |
|-----------|---------------------------------------------------|
| NUMBER    | Stores numeric values.                            |
| VARCHAR2  | Stores variable-length character data.            |
| CHAR      | Stores fixed-length character data.               |
| DATE      | Stores date and time values.                      |
| BOOLEAN   | Stores Boolean values (TRUE, FALSE, NULL).        |
| LONG      | Stores large character data (less commonly used). |
| RAW       | Stores binary data.                               |
| BLOB      | Stores large binary data.                         |
| CLOB      | Stores large character data.                      |
| BFILE     | Stores binary file data (external file).          |

## 9. NUMBER Data Type

- Used to store numeric values (integer or decimal).
- Syntax :

```
variable_name NUMBER(size, scale);
```

  - size → total number of digits
  - scale → number of digits after decimal point

Example :

```
DECLARE
v_id NUMBER(5); -- integer
v_sal NUMBER(8,2); -- decimal
BEGIN
v_id := 101;
v_sal := 12345.67;
DBMS_OUTPUT.PUT_LINE('ID = ' || v_id);
DBMS_OUTPUT.PUT_LINE('Salary = ' || v_sal);
END;
```

## 10. VARCHAR2 and CHAR Data Types

- Used to store character (string) values.
- VARCHAR2** → variable length (stores only actual data).
- CHAR** → fixed length (pads spaces if required).

Syntax :

```
variable_name VARCHAR2(size);
variable_name CHAR(size);
```

Example :

```
DECLARE
v_name VARCHAR2(20);
v_code CHAR(5);
BEGIN
v_name := 'FORSKO';
v_code := 'CS101';
DBMS_OUTPUT.PUT_LINE('Name = ' || v_name);
DBMS_OUTPUT.PUT_LINE('Code = ' || v_code);
END;
```

Output :

Name = FORSKO  
Code = CS101

## 11. DATE Data Type

- Used to store date and time values.
- Default format is DD-MON-YYYY.
- It also stores time (hours, minutes, seconds).

Syntax :

```
variable_name DATE;
```

Example :

```
DECLARE
v_join_date DATE;
BEGIN
v_join_date := TO_DATE('03-10-2026',
'DD-MM-YYYY');
DBMS_OUTPUT.PUT_LINE('Join Date = ' ||
TO_CHAR(v_join_date, 'DD-MON-YYYY'));
END;
```

Output :

Join Date = 03-OCT-2026

## 12. BOOLEAN Data Type

- Used to store Boolean values.
- It can have three values : TRUE, FALSE, NULL.
- Mostly used in conditions and control statements.

Syntax :

```
variable_name BOOLEAN;
```

Example :

```
DECLARE
v_flag BOOLEAN;
BEGIN
v_flag := TRUE;
IF v_flag THEN
DBMS_OUTPUT.PUT_LINE('Flag is TRUE');
END IF;
END;
```

Output :

Flag is TRUE



# Block Structure Of PL/SQL

## 1. Introduction

- A PL/SQL block is a logical unit of code that consists of a group of statements.
- It is the basic unit of execution in PL/SQL.
- Every PL/SQL block follows a specific structure.
- The block structure allows us to declare variables, write executable statements and handle exceptions in a single unit.
- In PL/SQL, we can write anonymous blocks (without name) or named blocks (procedures, functions, packages, etc.).

## 2. Structure of a PL/SQL Block



## 3. Explanation of Each Section

- DECLARE Section (Optional)**
  - Used to declare variables, constants, cursors, user-defined exceptions, etc.
  - If there is no declaration, this section can be omitted.
- BEGIN Section (Mandatory)**
  - Contains the executable statements.
  - At least one executable statement must be present in this section.
- EXCEPTION Section (Optional)**
  - Used to handle runtime errors (exceptions).
  - If there is no exception handling, this section can be omitted.
- END; (Mandatory)**
  - Marks the end of the PL/SQL block.
  - Every PL/SQL block must end with END;

## 4. Syntax of a PL/SQL Block

```

DECLARE -- Optional section
-- Variable, constant, cursor, etc. declarations
variable_name data_type;
constant_name CONSTANT data_type := value;
cursor_name CURSOR ... ;

BEGIN -- Mandatory section
-- Executable statements
-- DML, queries, control statements, etc.

EXCEPTION -- Optional section
-- Exception handling statements
WHEN exception_name THEN
-- statements;

END; -- Block must end with END;
  
```

## 5. Example 1 : Simple PL/SQL Block (Without DECLARE and EXCEPTION)

```

BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello PL/SQL');
END;
/
  
```

Output :

Hello PL/SQL

## 6. Example 2 : PL/SQL Block with DECLARE

```

DECLARE
  v_name VARCHAR2(20);
  v_age NUMBER;
BEGIN
  v_name := 'FORSKO';
  v_age := 21;
  DBMS_OUTPUT.PUT_LINE('Name : ' || v_name);
  DBMS_OUTPUT.PUT_LINE('Age : ' || v_age);
END;
/
  
```

Output :

Name : FORSKO  
Age : 21

## 7. Example 3 : PL/SQL Block with EXCEPTION

```

DECLARE
  v_num NUMBER := 10;
  v_res NUMBER;
BEGIN
  v_res := 100 / v_num;
  DBMS_OUTPUT.PUT_LINE('Result : ' || v_res);
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
/
  
```

Output :

Result : 10

## 8. Example 4 : Complete PL/SQL Block (DECLARE, BEGIN, EXCEPTION)

```

DECLARE
  v_num1 NUMBER := 20;
  v_num2 NUMBER := 0;
  v_result NUMBER;
BEGIN
  v_result := v_num1 / v_num2;
  DBMS_OUTPUT.PUT_LINE('Result : ' || v_result);
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Error : Division by zero is not allowed');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error : ' || SQLERRM);
END;
/
  
```

Output :

Error : Division by zero  
is not allowed

## 9. Important Points

- DECLARE section is optional, BEGIN section is mandatory and EXCEPTION section is optional.
- Every PL/SQL block must end with END;
- If DECLARE section is not required, we can start directly with BEGIN.
- We can declare multiple variables in the DECLARE section.
- The EXCEPTION section is used to handle runtime errors and prevent abnormal termination of the program.
- A PL/SQL block can contain SQL statements, PL/SQL statements, control structures (IF, LOOP, etc.) and exception handling.
- The block is executed as a single unit.

## 10. Types of PL/SQL Blocks

| Type                      | Description                                           | Example                                   |
|---------------------------|-------------------------------------------------------|-------------------------------------------|
| 1 Anonymous Block         | A block without a name. Used for temporary execution. | DECLARE ... BEGIN ... END;                |
| 2 Named Block (Procedure) | A block with a name that can be stored and called.    | CREATE OR REPLACE PROCEDURE proc_name ... |
| 3 Named Block (Function)  | A block that returns a value.                         | CREATE OR REPLACE FUNCTION func_name ...  |
| 4 Named Block (Package)   | A collection of related procedures and functions.     | CREATE OR REPLACE PACKAGE pkg_name ...    |



# Exception Handling in PL/SQL

## 1. Introduction to Exception Handling

- An exception is a runtime error or an unexpected event that occurs during the execution of a PL/SQL block.
- Exception handling is used to handle these errors gracefully without abnormal termination of the program.
- It allows us to detect the error, take appropriate action and continue the execution of the program.
- Exception handling improves the reliability and robustness of PL/SQL programs.

## 2. Types of Exceptions

| Type                             | Description                                                                                                    |
|----------------------------------|----------------------------------------------------------------------------------------------------------------|
| 1 Predefined (Oracle) Exceptions | Built-in exceptions defined by Oracle (e.g. <b>NO_DATA_FOUND</b> , <b>TOO_MANY_ROWS</b> , <b>ZERO_DIVIDE</b> ) |
| 2 User-Defined Exceptions        | Exceptions that we define using the <b>EXCEPTION</b> keyword.                                                  |
| 3 System-Defined Exceptions      | Raised by Oracle automatically for runtime errors (e.g. invalid data, constraint violation, etc.).             |

## 3. Common Predefined Exceptions

| Exception Name          | Description                                                     |
|-------------------------|-----------------------------------------------------------------|
| <b>NO_DATA_FOUND</b>    | Raised when a SELECT statement returns no rows.                 |
| <b>TOO_MANY_ROWS</b>    | Raised when a SELECT statement returns more than one row.       |
| <b>ZERO_DIVIDE</b>      | Raised when division by zero occurs.                            |
| <b>DUP_VAL_ON_INDEX</b> | Raised when insert/update violates a unique constraint.         |
| <b>VALUE_ERROR</b>      | Raised when a data conversion error occurs.                     |
| <b>INVALID_CURSOR</b>   | Raised when a cursor is used incorrectly.                       |
| <b>OTHERS</b>           | Catches all other exceptions (not recommended as sole handler). |

## 4. Syntax of Exception Handling

```

DECLARE
    -- Declarations (variables, constants,
    user-defined exceptions, etc.)

BEGIN
    -- Executable statements

EXCEPTION
    -- Exception handlers
    WHEN exception_name THEN
        -- statements;
    WHEN OTHERS THEN
        -- statements;

END;
    
```

## 5. Example 1 : Handling Division by Zero (ZERO\_DIVIDE)

```

DECLARE
    v_num1 NUMBER := 10;
    v_num2 NUMBER := 0;
    v_result NUMBER;

BEGIN
    v_result := v_num1 / v_num2;
    DBMS_OUTPUT.PUT_LINE('Result : ' || v_result);

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Error : Division by zero
        is not allowed.');
```

**Output :**

Error : Division  
by zero is not  
allowed.

## 6. Example 2 : Handling NO\_DATA\_FOUND

```

DECLARE
    v_name VARCHAR2(20);

BEGIN
    SELECT name INTO v_name
    FROM Employee
    WHERE id = 100; -- assume no such id
    DBMS_OUTPUT.PUT_LINE('Name : ' || v_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No record found
        for the given ID.');
```

**Output :**

No record  
found for  
the given  
ID.

## 7. Example 3 : Handling TOO\_MANY\_ROWS

```

DECLARE
    v_name VARCHAR2(20);

BEGIN
    SELECT name INTO v_name
    FROM Employee
    WHERE dept = 'CSE'; -- returns multiple rows
    DBMS_OUTPUT.PUT_LINE('Name : ' || v_name);

EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one
        record found.');
```

**Output :**

More than  
one record  
found.

## 8. Example 4 : User-Defined Exception

```

DECLARE
    v_age NUMBER := -5;
    e_invalid_age EXCEPTION; -- user-defined exception

BEGIN
    IF v_age < 0 THEN
        RAISE e_invalid_age;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Age : ' || v_age);
    END IF;

EXCEPTION
    WHEN e_invalid_age THEN
        DBMS_OUTPUT.PUT_LINE('Error : Invalid age. Age
        cannot be negative.');
```

**Output :**

Error : Invalid  
age. Age cannot  
be negative.

## 9. Example 5 : Handling OTHERS Exception

```

DECLARE
    v_num NUMBER := 10;
    v_text VARCHAR2(10) := 'abc';
    v_result NUMBER;

BEGIN
    v_result := v_num + v_text; -- invalid operation
    DBMS_OUTPUT.PUT_LINE('Result : ' || v_result);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error
        occurred : ' || SQLERRM);

END;
```

**Output :**

An unexpected  
error occurred :  
ORA-01722:  
invalid number

## 10. Important Points

- Exceptions are handled in the **EXCEPTION** section of a PL/SQL block.
- Each exception handler starts with **WHEN** and ends with **THEN**.
- Multiple exception handlers can be used in a single block.
- OTHERS** exception should be used to handle all other unexpected errors.
- Exception handling prevents abnormal termination of the program.
- We can use user-defined exceptions using the **EXCEPTION** keyword and **RAISE** statement.
- It helps in writing reliable and robust PL/SQL programs.
- Always provide meaningful error messages for better debugging.



# Cursor in PL/SQL : Types of Cursors

## 1. Introduction to Cursor

- A cursor is a pointer to a context area that holds the result set of a SQL query.
- It allows us to process multiple rows one at a time in PL/SQL.
- Cursors are used when a query returns more than one row.
- It provides a way to fetch rows from a result set and perform operations on each row individually.
- Cursors are mainly useful in situations where we need to perform row-by-row processing, complex logic or data manipulation.

## 2. Features of Cursor

- 1 Used to process multiple rows returned by a query.
- 2 Provides row-by-row processing.
- 3 Helps to control fetching of rows.
- 4 Can be explicit or implicit.
- 5 Offers attributes to check the status of cursor (e.g., %FOUND, %NOTFOUND).
- 6 Useful when we need more control over the query and result set.

## 3. Cursor Attributes

| Attribute | Description                                                           |
|-----------|-----------------------------------------------------------------------|
| %FOUND    | Returns TRUE if the last fetch returned a row.                        |
| %NOTFOUND | Returns TRUE if the last fetch did not return a row.                  |
| %ROWCOUNT | Returns the number of rows fetched so far.                            |
| %ISOPEN   | Returns TRUE if the cursor is open, otherwise FALSE.                  |
| %CURRENT  | Returns TRUE if the cursor is positioned on a valid row (for update). |

## 4. Types of Cursors

- There are two main types of cursors in PL/SQL :
  - 1 **Implicit Cursor**
    - Automatically created by Oracle.
    - Used for single-row queries (SELECT INTO) or DML statements (INSERT, UPDATE, DELETE).
    - We do not need to declare, open, fetch or close an implicit cursor.
  - 2 **Explicit Cursor**
    - Manually declared by the programmer.
    - Used for multi-row queries.
    - Must be declared, opened, fetched and closed explicitly.
    - Provides more control over the query and result set.

## 5. Implicit Cursor

- Implicit cursor is automatically created and managed by Oracle.
- It is used for single-row queries (SELECT INTO) and DML statements (INSERT, UPDATE, DELETE).
- We cannot control the implicit cursor directly.
- Attributes like SQL%FOUND, SQL%NOTFOUND and SQL%ROWCOUNT are used to get information.

### 5. Example 1 : Using Implicit Cursor with SELECT INTO

```
DECLARE
    v_name VARCHAR2(20);
    v_sal NUMBER;
BEGIN
    SELECT name, salary INTO v_name, v_sal
    FROM Employee WHERE id = 1;
    DBMS_OUTPUT.PUT_LINE('Name : ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary : ' || v_sal);
END;
```

**Output :**

Name : Ali  
Salary : 30000

## 6. Explicit Cursor

- Explicit cursor is declared manually.
- Used for multi-row queries.
- We must declare, open, fetch and close the cursor.
- Provides more flexibility and control compared to implicit cursor.

**Syntax :**

```
DECLARE
    CURSOR cursor_name IS
        SELECT column1, column2, ...
        FROM table_name
        WHERE condition;
BEGIN
    OPEN cursor_name;
    LOOP
        FETCH cursor_name INTO variable_list;
        EXIT WHEN cursor_name%NOTFOUND;
        -- process the row
    END LOOP;
    CLOSE cursor_name;
END;
```

## 7. Example 2 : Explicit Cursor

```
DECLARE
    CURSOR emp_cur IS
        SELECT id, name, salary
        FROM Employee;
        v_id NUMBER;
        v_name VARCHAR2(20);
        v_salary NUMBER;
BEGIN
    OPEN emp_cur;
    LOOP
        FETCH emp_cur INTO v_id, v_name, v_salary;
        EXIT WHEN emp_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('ID : ' || v_id ||
            ', Name : ' || v_name ||
            ', Salary : ' || v_salary);
    END LOOP;
    CLOSE emp_cur;
END;
```

**Output :**

ID : 1, Name : Ali, Salary : 30000  
ID : 2, Name : Sara, Salary : 40000  
ID : 3, Name : Aman, Salary : 35000

## 8. Types of Explicit Cursors

| Type                            | Description                                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------|
| 1 Simple Cursor                 | Used to process query result set in a forward direction (one row at a time).                          |
| 2 Parameterized Cursor          | Accepts parameters to filter data.                                                                    |
| 3 Ref Cursor (Reference Cursor) | A cursor variable that can point to a query result set. It can be returned from procedures/functions. |
| 4 Scrollable Cursor (Advanced)  | Allows movement in both directions (forward and backward). (Not commonly used in basic PL/SQL).       |

## 9. Simple Cursor

- A simple cursor is used to fetch rows from a query without any parameters.
- It is the most commonly used explicit cursor.

**Example :**

```
DECLARE
    CURSOR c1 IS SELECT name FROM Employee;
    v_name VARCHAR2(20);
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO v_name;
        EXIT WHEN c1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name);
    END LOOP;
    CLOSE c1;
END;
```

**Output :**

Ali  
Sara  
Aman  
Neha

## 10. Parameterized Cursor

- A parameterized cursor accepts parameters at the time of opening the cursor.
- It is useful to filter data based on a condition.

**Example :**

```
DECLARE
    CURSOR c2 (p_dept VARCHAR2) IS
        SELECT name, salary FROM Employee;
        WHERE dept = p_dept;
        v_name VARCHAR2(20);
        v_salary NUMBER;
BEGIN
    OPEN c2('CSE');
    LOOP
        FETCH c2 INTO v_name, v_salary;
        EXIT WHEN c2%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name || ' - ' || v_salary);
    END LOOP;
    CLOSE c2;
END;
```

**Output :**

Ali - 30000

Aman - 35000

## 11. Ref Cursor (Reference Cursor)

- A reference cursor is a cursor variable that holds a result set.
- It can be returned from a function or procedure.
- It is mainly used to pass query results between PL/SQL blocks.

**Example :**

```
DECLARE
    TYPE emp_ref IS REF CURSOR;
    c emp_ref;
    v_name VARCHAR2(20);
BEGIN
    OPEN c FOR SELECT name FROM Employee;
    LOOP
        FETCH c INTO v_name;
        EXIT WHEN c%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name);
    END LOOP;
    CLOSE c;
END;
```

**Output :**

Ali  
Sara  
Aman  
Neha

## 12. Important Points

- 1 Implicit cursor is automatically created by Oracle and cannot be controlled.
- 2 Explicit cursor must be declared, opened, fetched and closed manually.
- 3 %FOUND, %NOTFOUND, %ROWCOUNT and %ISOPEN are attributes of explicit cursor (while SQL% attributes are for implicit cursor).
- 4 Simple cursor is used for basic multi-row query.
- 5 Parameterized cursor accepts parameters to filter data.
- 6 Ref cursor is a cursor variable, useful in procedures/functions.
- 7 Cursors help in row-by-row processing and provides more control over data.
- 8 Always close explicit cursor after use to release resources.



# Cursor : Attributes of Cursor

## 1. Introduction to Cursor Attributes

- Cursor attributes are predefined functions in PL/SQL that provide information about the status of a cursor.
- These attributes are used to control the flow of a program and check the state of the cursor during execution.
- Cursor attributes return BOOLEAN or INTEGER values.
- They are very useful in exception handling, loop control and writing reliable and robust programs.

## 2. List of Cursor Attributes

| Attribute | Meaning                                                                                |
|-----------|----------------------------------------------------------------------------------------|
| %FOUND    | Returns TRUE if the last fetch returned a row.                                         |
| %NOTFOUND | Returns TRUE if the last fetch did not return a row.                                   |
| %ROWCOUNT | Returns the number of rows fetched so far.                                             |
| %ISOPEN   | Returns TRUE if the cursor is open, otherwise FALSE.                                   |
| %CURRENT  | Returns TRUE if the cursor is positioned on a valid row (for UPDATE/WHERE CURRENT OF). |

## 3. Explanation of Each Attribute

- %FOUND**
  - Returns TRUE if the most recent FETCH statement returned a row.
  - Otherwise, it returns FALSE.
  - Used to check whether data is found or not.
- %NOTFOUND**
  - Returns TRUE if the most recent FETCH statement did not return a row.
  - It is the opposite of %FOUND.
  - Used to exit a loop when no more rows.
- %ROWCOUNT**
  - Returns the total number of rows fetched so far from the cursor.
  - The first fetch returns 1, second 2, and so on.
  - Used to count how many rows have been processed.

## 4. Explanation (Contd.)

- %ISOPEN**
  - Returns TRUE if the cursor is currently open.
  - Returns FALSE if the cursor is closed.
  - Used to check whether the cursor is open before performing operations like FETCH or CLOSE.
- %CURRENT**
  - Returns TRUE if the cursor is currently positioned on a valid row.
  - Used with WHERE CURRENT OF in UPDATE or DELETE statements.
  - For cursors that do not support WHERE CURRENT OF, it always returns FALSE.

## 5. Example 1 : Using %FOUND and %NOTFOUND

```

DECLARE
CURSOR c1 IS
SELECT name, salary FROM Employee;
v_name VARCHAR2(20);
v_salary NUMBER;
BEGIN
OPEN c1;
LOOP
FETCH c1 INTO v_name, v_salary;
IF c1%NOTFOUND THEN
DBMS_OUTPUT.PUT_LINE('No more records.');

```

### Output :

```

Name : Ali,
Salary : 30000
Name : Sara,
Salary : 40000
Name : Aman,
Salary : 35000
No more records.
```

## 6. Example 2 : Using %ROWCOUNT

```

DECLARE
CURSOR c1 IS
SELECT name FROM Employee;
v_name VARCHAR2(20);
BEGIN
OPEN c1;
LOOP
FETCH c1 INTO v_name;
EXIT WHEN c1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('Row No. ' || c1%ROWCOUNT
|| : = ' || v_name);
END LOOP;
CLOSE c1;
END;
```

### Output :

```

Row No. 1 : Ali
Row No. 2 : Sara
Row No. 3 : Aman
```

## 7. Example 3 : Using %ISOPEN

```

DECLARE
CURSOR c1 IS
SELECT name FROM Employee;
BEGIN
IF c1%ISOPEN THEN
DBMS_OUTPUT.PUT_LINE('Cursor is already open');

```

### Output :

```

Cursor is not open
Now cursor is open
Cursor is closed
```

## 8. Example 4 : Using %CURRENT (for UPDATE)

```

DECLARE
CURSOR c1 IS
SELECT emp_id, name, salary FROM Employee;
FOR UPDATE;
v_id NUMBER;
v_name VARCHAR2(20);
v_salary NUMBER;
BEGIN
OPEN c1;
FETCH c1 INTO v_id, v_name, v_salary;
IF c1%CURRENT THEN
UPDATE Employee
SET salary = salary * 1000
WHERE CURRENT OF c1;
DBMS_OUTPUT.PUT_LINE('Record updated for ' || v_name);
END IF;
CLOSE c1;
END;
```

### Output :

```

Record updated
for Ali
```

## 9. Example 5 : Complete Program Using All Attributes

```

DECLARE
CURSOR c1 IS
SELECT emp_id, name, salary FROM Employee;
v_id NUMBER;
v_name VARCHAR2(20);
v_salary NUMBER;
BEGIN
OPEN c1;
DBMS_OUTPUT.PUT_LINE('Cursor opened? ' || c1%ISOPEN);
LOOP
FETCH c1 INTO v_id, v_name, v_salary;
IF c1%FOUND THEN
DBMS_OUTPUT.PUT_LINE('Row ' || c1%ROWCOUNT ||
: = ' || v_name || - ' || v_salary);
ELSE
DBMS_OUTPUT.PUT_LINE('No more rows (NOTFOUND = TRUE)');

```

### Output :

```

Cursor opened? TRUE
Row 1 : Ali - 30000
Row 2 : Sara - 40000
Row 3 : Aman - 35000
No more rows
(NOTFOUND = TRUE)
Cursor open? FALSE
```

## 10. Important Points

- %FOUND** returns TRUE if the last FETCH statement returned a row.
- %NOTFOUND** returns TRUE if the last FETCH did not return a row.
- %ROWCOUNT** returns the number of rows fetched so far.
- %ISOPEN** returns TRUE if the cursor is open.
- %CURRENT** returns TRUE if the cursor is positioned on a valid row (mainly used with WHERE CURRENT OF).
- Cursor attributes are used to control program flow and handle different conditions.
- They are very useful in loops, exception handling and update/delete operations.
- Always use these attributes to write efficient and reliable PL/SQL programs.



# Cursor : Life Cycle of Cursor in PL/SQL

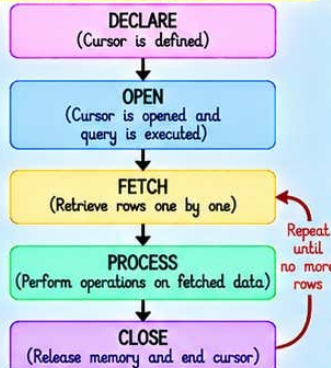
## 1. Introduction to Cursor Life Cycle

- A cursor goes through a sequence of steps from its creation to destruction. This sequence is called the life cycle of a cursor.
- A cursor is used to process multiple rows returned by a query.
- During its life cycle, the cursor is declared, opened, fetched, and finally closed.
- Understanding the life cycle helps us to use cursors properly and manage resources efficiently.

## 2. Stages in the Life Cycle of a Cursor

- 1 Declare the Cursor**  
→ Define the cursor with a SELECT statement.
- 2 Open the Cursor**  
→ Allocate memory and execute the query. The result set is created.
- 3 Fetch from the Cursor**  
→ Retrieve rows one by one from the result set into variables.
- 4 Process the Data**  
→ Perform required operations on the fetched data.
- 5 Close the Cursor**  
→ Release memory and invalidate the result set.

## 3. Diagram : Life Cycle of Cursor



## 4. Explanation of Each Stage

- 1 DECLARE**
  - Cursor is defined with a SELECT statement.
  - No memory is allocated at this stage.
- 2 OPEN**
  - Memory is allocated for the cursor.
  - Query is executed.
  - The result set is created and the cursor is now active.
- 3 FETCH**
  - Each call to FETCH retrieves one row from the result set.
  - The row is stored in variables.
  - When no more rows are available, %NOTFOUND becomes TRUE.
- 4 PROCESS**
  - Perform the required operations (e.g., display, calculation, update, etc.) on the fetched data.
- 5 CLOSE**
  - Releases the memory allocated for the cursor.
  - The result set is destroyed.
  - Cursor is no longer active and cannot be used unless it is opened again.

## 5. Cursor Life Cycle in Detail

| Stage          | Description                                   | What Happens                                                      | Cursor Status            | Attributes                                                             |
|----------------|-----------------------------------------------|-------------------------------------------------------------------|--------------------------|------------------------------------------------------------------------|
| <b>Declare</b> | Cursor is defined in the declaration section. | SELECT statement is associated with the cursor.                   | Inactive (not allocated) | %ISOPEN = FALSE                                                        |
| <b>Open</b>    | Cursor is opened in the executable section.   | Memory is allocated and query is executed. Result set is created. | Active                   | %ISOPEN = TRUE<br>%FOUND = FALSE<br>%NOTFOUND = FALSE<br>%ROWCOUNT = 0 |
| <b>Fetch</b>   | Retrieve rows one by one.                     | Each FETCH gets the next row from the result set into variables.  | Active                   | %FOUND, %NOTFOUND and %ROWCOUNT updated after each fetch.              |
| <b>Process</b> | Perform required operations.                  | Use the fetched data for display, calculation, etc.               | Active                   | Attributes reflect the current row.                                    |
| <b>Close</b>   | Cursor is closed.                             | Memory is released and result set is destroyed.                   | Inactive                 | %ISOPEN = FALSE                                                        |

## 6. Cursor Attributes During Life Cycle

| Attribute        | Meaning                                                                                | Value          |
|------------------|----------------------------------------------------------------------------------------|----------------|
| <b>%ISOPEN</b>   | Returns TRUE if the cursor is currently open.                                          | TRUE / FALSE   |
| <b>%FOUND</b>    | Returns TRUE if the last FETCH returned a row.                                         | TRUE / FALSE   |
| <b>%NOTFOUND</b> | Returns TRUE if the last FETCH did not return a row.                                   | TRUE / FALSE   |
| <b>%ROWCOUNT</b> | Returns the number of rows fetched so far.                                             | 0, 1, 2, ... n |
| <b>%CURRENT</b>  | Returns TRUE if the cursor is positioned on a valid row (for UPDATE/WHERE CURRENT OF). | TRUE / FALSE   |

## 7. Example : Complete Life Cycle of an Explicit Cursor

```

DECLARE
CURSOR c_emp IS
SELECT emp_id, emp_name, salary FROM Employee;
v_id NUMBER;
v_name VARCHAR2(20);
v_salary NUMBER;
BEGIN
-- Open the cursor
OPEN c_emp;
-- Fetch and process rows
LOOP
FETCH c_emp INTO v_id, v_name, v_salary;
EXIT WHEN c_emp%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('ID : ' || v_id || ' Name : ' || v_name || ' Salary : ' || v_salary);
END LOOP;
-- Close the cursor
CLOSE c_emp;
END;
    
```

Output :

```

ID : 1, Name : Ali, Salary : 30000
ID : 2, Name : Sara, Salary : 40000
ID : 3, Name : Aman, Salary : 35000
    
```

## 8. Example : Checking Cursor Attributes

```

DECLARE
CURSOR c_emp IS
SELECT emp_id, emp_name FROM Employee;
v_id NUMBER;
v_name VARCHAR2(20);
BEGIN
OPEN c_emp;
DBMS_OUTPUT.PUT_LINE('Cursor Open? ' || c_emp%ISOPEN);
FETCH c_emp INTO v_id, v_name;
IF c_emp%FOUND THEN
DBMS_OUTPUT.PUT_LINE('Fetched: ' || v_name);
END IF;
DBMS_OUTPUT.PUT_LINE('Rows Fetched: ' || c_emp%ROWCOUNT);
FETCH c_emp INTO v_id, v_name;
IF c_emp%NOTFOUND THEN
DBMS_OUTPUT.PUT_LINE('No more rows');
END IF;
CLOSE c_emp;
DBMS_OUTPUT.PUT_LINE('Cursor Open? ' || c_emp%ISOPEN);
END;
    
```

Output :

```

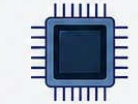
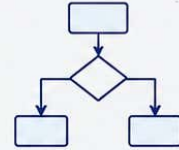
Cursor Open? TRUE
Fetched: Ali
Rows Fetched: 1
Fetched: Sara
Rows Fetched: 2
No more rows
Cursor Open? FALSE
    
```

## 9. Important Points

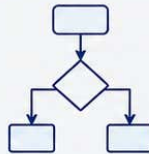
- Cursor life cycle consists of **DECLARE**, **OPEN**, **FETCH**, **PROCESS** and **CLOSE**.
- Cursor is inactive after declaration and becomes active only after opening.
- FETCH** retrieves one row at a time.
- %NOTFOUND** becomes TRUE when no more rows are available.
- Cursor must be closed to release memory and database resources.
- We can check the status of a cursor using cursor attributes.
- If a cursor is not closed, it may lead to resource leakage.
- A cursor can be reopened after closing it.



1010101010  
0101010101  
1010101010  
0101010101



10110  
01001  
10101



# Unit - IV



# Procedure in PL/SQL : Introduction, Importance, Types, Advantages & Disadvantages

## 1. Introduction to Procedure

- A procedure is a named PL/SQL block that is stored in the database and can be executed whenever needed.
- It contains a set of SQL and PL/SQL statements to perform a specific task.
- It is created using the CREATE PROCEDURE statement.
- A procedure may or may not return a value.
- It helps in modular programming by reusing code.

### Syntax :

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_list)
IS / AS
-- Declarations
BEGIN
-- Executable statements
EXCEPTION
-- Exception handling
END;
/
```

## 2. Example of a Simple Procedure

```
CREATE OR REPLACE PROCEDURE show_message
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello, This is a Procedure!');
END;
/
```

### Explanation :

- show\_message is the name of the procedure.
- It has no parameters.
- It simply prints a message using DBMS\_OUTPUT.
- The procedure is stored in the database and can be executed anytime.

### Execution :

- After creating, execute it using:

```
EXEC show_message;
```

Output : Hello, This is a Procedure!

## 3. Importance of Procedure

- 1 Code Reusability**  
A procedure can be executed multiple times, saving development time.
- 2 Modularity**  
Divides a large program into smaller and manageable blocks.
- 3 Reduced Network Traffic**  
Logic is stored in the database, so only the procedure call is sent over the network.
- 4 Better Performance**  
Stored and compiled in the database, so execution is faster.
- 5 Security**  
Direct access to tables can be restricted and users can be given permission only to execute procedures.
- 6 Maintainability**  
Changes can be done in one place without modifying multiple programs.
- 7 Data Integrity**  
Business rules can be implemented consistently.
- 8 Abstraction**  
Hides the complex logic from end users.

## 4. Types of Procedures

| Type                              | Description                                            |
|-----------------------------------|--------------------------------------------------------|
| <b>1 System-Defined Procedure</b> | Provided by Oracle. (e.g., DBMS_OUTPUT.PUT_LINE)       |
| <b>2 User-Defined Procedure</b>   | Created by the user to perform a specific task.        |
| <b>3 Parameterless Procedure</b>  | A procedure that does not accept any parameters.       |
| <b>4 Parameterized Procedure</b>  | A procedure that accepts parameters (IN, OUT, IN OUT). |
| <b>5 Stored Procedure</b>         | A procedure stored in the database for repeated use.   |
| <b>6 Recursive Procedure</b>      | A procedure that calls itself.                         |

### Example : Parameterized Procedure

```
CREATE OR REPLACE PROCEDURE display_employee
(p_id NUMBER, p_name VARCHAR2)
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('ID : ' || p_id);
    DBMS_OUTPUT.PUT_LINE('Name : ' || p_name);
END;
/
```

### Execution :

```
EXEC display_employee
(101, 'Ali');
```

## 5. Advantages of Procedure

- 1 Code Reusability**  
Can be executed multiple times.
- 2 Improved Performance**  
Stored and compiled in the database.
- 3 Reduced Network Traffic**  
Only the procedure call is sent to the server.
- 4 Better Security**  
Access to tables can be restricted.
- 5 Modular Programming**  
Large programs are divided into smaller blocks.
- 6 Easier Maintenance**  
Changes can be made in one place.
- 7 Data Integrity**  
Business rules can be enforced consistently.
- 8 Abstraction**  
Hides complex implementation from end users.

## 6. Disadvantages of Procedure

- 1 Complexity**  
Writing and debugging procedures can be difficult for beginners.
- 2 Initial Development Time**  
Takes more time to design and develop.
- 3 Database Dependency**  
Code is tied to a specific database (like Oracle).
- 4 Limited Portability**  
Not easily transferable between different database systems.
- 5 Consumes Database Resources**  
Stored procedures use memory and space in the database.
- 6 Error Handling Needed**  
Proper exception handling must be written to manage runtime errors.

## 7. Example : Procedure with Parameters

```
CREATE OR REPLACE PROCEDURE calculate_salary
(p_basic IN NUMBER;
p_bonus IN NUMBER;
p_total OUT NUMBER)
IS
BEGIN
    p_total := p_basic + p_bonus;
END;
/
```

### Execution :

```
DECLARE
v_total NUMBER;
BEGIN
    calculate_salary(30000, 5000, v_total);
    DBMS_OUTPUT.PUT_LINE('Total Salary : ' || v_total);
END;
/
```

Output : Total Salary : 35000

## 8. Summary

- A procedure is a named block of PL/SQL code stored in the database.
- It helps in code reusability, better performance, security and maintainability.
- Types of procedures include system-defined, user-defined, parameterless, parameterized, stored and recursive procedures.
- Procedures have many advantages such as modularity, reusability and data integrity.
- However, they also have some disadvantages like complexity, initial development time and database dependency.
- Procedures are an important feature of PL/SQL and are widely used in real-world applications.



# Function in PL/SQL : Introduction, Importance, Types, Advantages & Disadvantages

## 1. Introduction to Function

- A function is a named PL/SQL block that performs a specific task and always returns a single value.
- It contains a set of SQL and PL/SQL statements and can be executed whenever needed.
- Functions are created using the CREATE FUNCTION statement.
- A function must have a RETURN type and must return a value using the RETURN statement.
- Functions can be called from PL/SQL blocks, SQL statements (SELECT) and from other functions or procedures.

## 2. Syntax of Function

```
CREATE OR REPLACE FUNCTION function_name
(parameter_list)
RETURN data_type
IS / AS
-- Declarations
BEGIN
-- Executable statements
-- RETURN value;
EXCEPTION
-- Exception handling
END;
/
```

### Explanation :

- function\_name: Name of the function.
- parameter\_list: Input parameters (optional).
- data\_type: Return type (e.g., NUMBER, VARCHAR2).
- RETURN: Specifies the value to be returned.

## 3. Example : Simple Function

```
CREATE OR REPLACE FUNCTION add_numbers
(a NUMBER, b NUMBER)
RETURN NUMBER
IS
result NUMBER;
BEGIN
result := a + b;
RETURN result;
END;
/
```

### Execution :

```
DECLARE
v_sum NUMBER;
BEGIN
v_sum := add_numbers(10, 20);
DBMS_OUTPUT.PUT_LINE('Sum : ' || v_sum);
END;
/
```

**Output :** Sum : 30

## 4. Types of Functions

| Type                              | Description                                                                                                                              |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 1 Built-in Functions (Predefined) | Provided by Oracle. Used for common operations like string, number, date, conversion, etc. (e.g., UPPER, LENGTH, SYSDATE, NVL, TO_CHAR). |
| 2 User-Defined Functions          | Created by the user to perform a specific task as per requirement.                                                                       |
| 3 Functions with Parameters       | Accept input parameters and return a value.                                                                                              |
| 4 Functions without Parameters    | Do not accept any input parameters but still return a value.                                                                             |
| 5 Scalar Functions                | Return a single value of a data type (NUMBER, VARCHAR2, DATE, etc.). (PL/SQL functions always return a single scalar value).             |

## 5. Example : Function with Parameters

```
CREATE OR REPLACE FUNCTION calculate_area
(radius NUMBER)
RETURN NUMBER
IS
area NUMBER;
BEGIN
area := 3.1416 * radius * radius;
RETURN area;
END;
/
```

### Execution :

```
BEGIN
DBMS_OUTPUT.PUT_LINE('Area : ' ||
calculate_area(5));
END;
/
```

**Output :** Area : 78.54

## 6. Example : Function without Parameters

```
CREATE OR REPLACE FUNCTION get_greeting
RETURN VARCHAR2
IS
BEGIN
RETURN 'Welcome to PL/SQL Functions';
END;
/
```

### Execution :

```
BEGIN
DBMS_OUTPUT.PUT_LINE(get_greeting);
END;
/
```

**Output :** Welcome to PL/SQL Functions

## 7. Example : Using Built-in Function in SQL

```
SELECT UPPER('oracle') AS upper_case,
LENGTH('PLSQL') AS length,
SYSDATE AS current_date,
NVL(NULL, 'Default') AS nvl_example
FROM dual;
```

### Output :

| UPPER_CASE | LENGTH | CURRENT_DATE | NVL_EXAMPLE |
|------------|--------|--------------|-------------|
| ORACLE     | 5      | 03-OCT-2026  | Default     |

## 8. Example : Calling Function from SQL

```
CREATE OR REPLACE FUNCTION square_number
(num NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN num * num;
END;
/
```

### Execution :

```
SELECT square_number(6) AS square
FROM dual;
```

**Output :** SQUARE

36

## 9. Importance of Functions

- Code Reusability**  
Can be called multiple times.
- Modularity**  
Divides a large program into smaller blocks.
- Easier Maintenance**  
Changes can be done in one place.
- Improved Productivity**  
Reduces development time.
- Better Organization**  
Keeps code structured and readable.
- Can be Used in SQL**  
Functions can be called in SQL SELECT statements.
- Business Logic Implementation**  
Helps in implementing complex business rules.

## 10. Advantages of Functions

- Code reusability and modularity.
- Always returns a value, which can be used in expressions or SQL queries.
- Reduces code duplication.
- Improves program readability and maintainability.
- Can be called from SQL, PL/SQL and other programs.
- Helps in implementing complex business logic.
- Can be used with parameters (Flexible).
- Improves productivity and reduces errors.

## 11. Disadvantages of Functions

- Always returns only a single value (cannot return multiple values).
- More complex to write compared to procedures for tasks that do not return a value.
- If not written properly, it may impact performance.
- Cannot be used to perform multiple operations that require no return value (in such cases procedures are better).
- Debugging can be more difficult for beginners.
- Excessive use may lead to performance overhead if not optimized.

## 12. Summary

- A function is a named block of PL/SQL code that always returns a single value.
- It is created using CREATE FUNCTION.
- Functions can be built-in (Oracle provided) or user-defined.
- Types include built-in functions, user-defined functions, functions with parameters, functions without parameters and scalar functions.
- Functions are important for code reusability, modularity, better maintenance and productivity.
- Advantages include reusability, readability, flexibility and can be used in SQL.
- Disadvantages include single return value, complexity and possible performance overhead.



# Triggers in PL/SQL : Introduction, Importance, Types, Advantages & Disadvantages

## 1. Introduction to Triggers

- A trigger is a special type of PL/SQL block that is automatically executed (fired) when a specified event occurs on a table or view.
- Triggers are associated with a database object (table or view).
- They are executed automatically by the database server; we do not call them explicitly.
- Triggers are mainly used to enforce business rules, maintain data integrity and perform automatic actions such as logging, auditing or validations.
- A trigger can be created using the CREATE TRIGGER statement.

## 2. Syntax of Trigger

```
CREATE OR REPLACE TRIGGER trigger_name
[BEFORE | AFTER | INSTEAD OF]
[INSERT OR UPDATE OR DELETE]
ON table_name
[FOR EACH ROW]
DECLARE
    -- Declarations (optional)
BEGIN
    -- Executable statements
END;
```

### Explanation :

- BEFORE / AFTER / INSTEAD OF : When the trigger will fire.
- INSERT / UPDATE / DELETE : Event on which trigger will fire.
- FOR EACH ROW : Executes once for each row affected (Row-level trigger).
- DECLARE : Optional declaration section.
- BEGIN ... END : PL/SQL block.

## 3. Example : Simple Trigger

```
CREATE OR REPLACE TRIGGER emp_log_trg
AFTER INSERT ON employee
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('New employee
added : ' || :NEW.emp_id || ' - ' ||
:NEW.emp_name);
END;
```

### Explanation :

- This trigger will fire after a new row is inserted into the employee table.
- :NEW refers to the new values being inserted.
- FOR EACH ROW means it executes for every inserted row.

### Output :

New employee added : 101 - Ali

## 4. Types of Triggers

| Type                      | Description                                                                                           |
|---------------------------|-------------------------------------------------------------------------------------------------------|
| 1 DML Trigger             | Fired on DML operations (INSERT, UPDATE, DELETE) on a table or view.                                  |
| 2 DDL Trigger             | Fired on DDL operations (CREATE, ALTER, DROP) at database level or schema level.                      |
| 3 Database Trigger        | Fired on database level events like startup, shutdown, logon, logoff, etc.                            |
| 4 Instead of Trigger      | Defined on a view and executes instead of the triggering DML operation. Used to make views updatable. |
| 5 Row-level Trigger       | Fires once for each row affected by the DML operation. (Uses FOR EACH ROW)                            |
| 6 Statement-level Trigger | Fires once for the whole statement (not for each row). (Does not use FOR EACH ROW)                    |

## 5. Examples of Different Types of Triggers

### 1 DML Trigger (AFTER INSERT)

```
CREATE OR REPLACE TRIGGER dept_insert_trg
AFTER INSERT ON department
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Department
added : ' || :NEW.dept_name);
END;
```

### 2 DDL Trigger

```
CREATE OR REPLACE TRIGGER ddl_audit_trg
AFTER CREATE OR ALTER OR DROP ON SCHEMA
BEGIN
    DBMS_OUTPUT.PUT_LINE('DDL operation
performed by ' || USER || ' on '
|| ORA_DICT_OBJ_NAME);
END;
```

### 3 INSTEAD OF Trigger (on a View)

```
CREATE OR REPLACE TRIGGER view_insert_trg
INSTEAD OF INSERT ON emp_view
BEGIN
    INSERT INTO employee(emp_id, emp_name)
VALUES(:NEW.emp_id, :NEW.emp_name);
END;
```

### 4 Row-level vs Statement-level

| Row-level Trigger                                                                                                                                        | Statement-level Trigger                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| CREATE OR REPLACE TRIGGER row_trg<br>AFTER UPDATE ON employee<br>FOR EACH ROW<br>BEGIN<br>DBMS_OUTPUT.PUT_LINE('Row updated : '    :NEW.emp_id);<br>END; | CREATE OR REPLACE TRIGGER stmt_trg<br>AFTER UPDATE ON employee<br>BEGIN<br>DBMS_OUTPUT.PUT_LINE('Employee table updated');<br>END; |

## 6. Importance of Triggers

- Enforces business rules automatically.
- Maintains data integrity and consistency.
- Helps in auditing and logging changes in data.
- Prevents invalid data from being inserted, updated or deleted.
- Reduces manual work and ensures that important actions are always performed.
- Useful for maintaining related data (such as updating summary tables).

## 7. Advantages of Triggers

- Automatic Execution : No need to call explicitly.
- Maintains Data Integrity : Enforces rules and validations.
- Audit and Logging : Keeps track of changes.
- Improves Security : Restricts unauthorized operations.
- Helps in Maintaining Derived Data : Automatically updates related tables.
- Centralized Logic : Business rules are stored in the database.
- Can Be Used with Views : Instead of triggers make views updatable.

## 8. Disadvantages of Triggers

- Complexity : Can make the system harder to understand and debug.
- Performance Overhead : Executes automatically, which may slow down DML operations.
- Hidden Execution : Runs in the background, so it is not always visible to the user.
- Maintenance Difficulty : Hard to manage in large applications.
- Debugging is Difficult : Errors can be tricky to find.
- Possible Unintended Effects : May cause unexpected behavior if not written carefully.
- Not Suitable for Complex Processing : Heavy logic in triggers can affect performance.

## 9. Summary

- A trigger is a special PL/SQL block that executes automatically when a specified event occurs.
- Triggers are created using the CREATE TRIGGER statement.
- Types include DML, DDL, Database, Instead of, Row-level and Statement-level triggers.
- Triggers are important for enforcing business rules, maintaining data integrity and auditing.
- Advantages include automatic execution, data integrity, security and logging.
- Disadvantages include complexity, performance overhead and difficulty in debugging.
- Triggers should be used carefully and only when necessary.



# Creation of Procedure, Functions in PL/SQL

## 1. Creation of Procedure

- A procedure is a named PL/SQL block that performs a specific task.
- It is created using the CREATE PROCEDURE statement.
- A procedure can have parameters (IN, OUT, IN OUT) or no parameters.
- It does not return a value explicitly.
- It can be executed using the EXEC command or by calling it from another PL/SQL block.

## 2. Syntax of Creating a Procedure

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_list)
IS / AS
-- Declarations (optional)
BEGIN
-- Executable statements
EXCEPTION
-- Exception handling (optional)
END;
```

## 7. Creation of Function

- A function is a named PL/SQL block that performs a specific task and always returns a single value.
- It is created using the CREATE FUNCTION statement.
- A function must have a RETURN type.
- It can have parameters or no parameters.
- Functions can be called from PL/SQL blocks, SQL statements (SELECT) and from other functions or procedures.

## 8. Syntax of Creating a Function

```
CREATE OR REPLACE FUNCTION function_name
(parameter_list)
RETURN data_type
IS / AS
-- Declarations (optional)
BEGIN
-- Executable statements
RETURN value;
EXCEPTION
-- Exception handling (optional)
END;
```

## 4. Example 1 : Procedure without Parameters

```
CREATE OR REPLACE PROCEDURE show_message
IS
BEGIN
DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL Procedure!');
END;
```

**Explanation :**

- This procedure does not take any parameters.
- It simply prints a message using DBMS\_OUTPUT.

### Calling the Procedure :

```
BEGIN
show_message;
END;
```

### Output :

Welcome to PL/SQL Procedure!

## 5. Example 2 : Procedure with Parameters (IN and OUT)

```
CREATE OR REPLACE PROCEDURE calc_area
(radius IN NUMBER, area OUT NUMBER)
IS
BEGIN
area := 3.1416 * radius * radius;
END;
```

### Calling the Procedure :

```
DECLARE
v_area NUMBER;
BEGIN
calc_area(5, v_area);
DBMS_OUTPUT.PUT_LINE('Area : ' || v_area);
END;
```

### Output :

Area : 78.54

## 9. Types of Parameters in Function

| Parameter Type | Description                                      |
|----------------|--------------------------------------------------|
| IN (default)   | Used to pass values to the function. (read only) |
| OUT            | Not allowed in functions.                        |
| IN OUT         | Not allowed in functions.                        |

## 10. Example 1 : Function without Parameters

```
CREATE OR REPLACE FUNCTION get_message
RETURN VARCHAR2
IS
BEGIN
RETURN 'Welcome to PL/SQL Functions!';
END;
```

### Calling the Function :

```
DECLARE
v_msg VARCHAR2(50);
BEGIN
v_msg := get_message;
DBMS_OUTPUT.PUT_LINE(v_msg);
END;
```

### Output :

Welcome to PL/SQL Functions!

## 6. Example 3 : Procedure with IN OUT Parameter

```
CREATE OR REPLACE PROCEDURE increment_value
(num IN OUT NUMBER)
IS
BEGIN
num := num + 10;
END;
```

### Calling the Procedure :

```
DECLARE
v_num NUMBER := 20;
BEGIN
increment_value(v_num);
DBMS_OUTPUT.PUT_LINE('After increment : ' || v_num);
END;
```

### Output :

After increment : 30

## 11. Example 2 : Function with Parameters

```
CREATE OR REPLACE FUNCTION add_numbers
(a NUMBER, b NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN a + b;
END;
```

### Calling the Function :

```
DECLARE
result NUMBER;
BEGIN
result := add_numbers(10, 20);
DBMS_OUTPUT.PUT_LINE('Sum : ' || result);
END;
```

### Output :

Sum : 30

## 12. Example 3 : Function for Area of Circle

```
CREATE OR REPLACE FUNCTION circle_area
(radius NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN 3.1416 * radius * radius;
END;
```

### Calling the Function :

```
DECLARE
area NUMBER;
BEGIN
area := circle_area(5);
DBMS_OUTPUT.PUT_LINE('Area : ' || area);
END;
```

### Output :

Area : 78.54

## 13. Example 4 : Function used in SQL

```
CREATE OR REPLACE FUNCTION square_num
(num NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN num * num;
END;
```

### Using in SQL :

```
SELECT num, square_num(num)
AS square_value
FROM (
SELECT 2 AS num FROM dual UNION ALL
SELECT 3 FROM dual UNION ALL
SELECT 4 FROM dual
);
```

### Output :

| NUM | SQUARE_VALUE |
|-----|--------------|
| 2   | 4            |
| 3   | 9            |
| 4   | 16           |



# Backup and Recovery : Introduction

## 1. Introduction to Backup and Recovery

- Backup and Recovery is a crucial part of Database Management System (DBMS).
- It is a process of creating a copy of the database and restoring it when the original data is lost, corrupted or unavailable.
- Backup is the process of making a copy of data and storing it in a safe location.
- Recovery is the process of restoring the data from the backup copy so that the database can be brought back to a consistent state.
- It helps in ensuring data safety, minimizing data loss and maintaining the availability of the database.

## 2. Need of Backup and Recovery

- **System Failures** : Hardware or software failure may cause data loss.
- **Human Errors** : Accidental deletion or incorrect updates.
- **Natural Disasters** : Fire, flood, earthquake, etc.
- **Security Threats** : Hacking, virus attack or malicious activities.
- **Media Failure** : Storage devices may get damaged.
- **Data Corruption** : Due to power failure or other unexpected reasons.
- **Business Continuity** : Ensures that the organization can continue its operations without major data loss.

## 3. Objectives of Backup and Recovery

- 1 **Data Protection** : Protects data from loss or corruption.
- 2 **Data Availability** : Ensures data can be restored when needed.
- 3 **Business Continuity** : Helps the organization continue operations.
- 4 **Minimize Downtime** : Reduces the time required to restore the system.
- 5 **Maintain Consistency** : Restores database to a consistent and valid state.
- 6 **Disaster Recovery** : Helps in recovering database after catastrophic events.
- 7 **Compliance** : Necessary for legal and regulatory requirements.

## 4. Basic Concepts

- **Backup** : A copy of database stored in a separate location.
- **Restore** : The process of bringing back the data from backup.
- **Recovery** : The complete process of restoring the database to a consistent state.
- **Recovery Time** : The time required to restore the database.
- **Recovery Point** : The point in time to which the data is restored.
- **Consistency** : The state where data remains correct and valid after recovery.
- **Disaster** : An event that causes serious data loss (e.g., hardware failure, natural disaster, cyber attack).

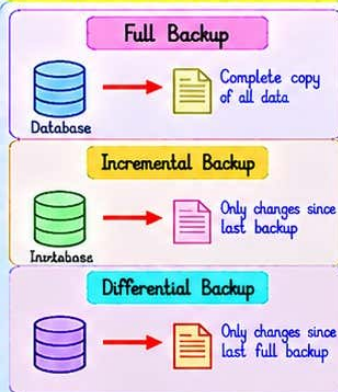
## 5. Types of Failures Requiring Recovery

| Type of Failure              | Description                                                         |
|------------------------------|---------------------------------------------------------------------|
| 1 <b>Transaction Failure</b> | A transaction fails due to logical error, deadlock or system abort. |
| 2 <b>System Failure</b>      | Entire system stops working due to power failure or hardware crash. |
| 3 <b>Media Failure</b>       | Storage media like disk gets damaged.                               |
| 4 <b>Human Error</b>         | Accidental deletion or incorrect update of data.                    |
| 5 <b>Natural Disaster</b>    | Fire, flood, earthquake, etc.                                       |
| 6 <b>Malicious Attack</b>    | Virus attack, hacking, ransomware, etc.                             |

## 6. Types of Backup

- 1 **Full Backup**
  - Complete copy of the entire database.
  - Takes more storage space and time.
  - Restoration is simple and fast.
  - Example : Full database backup once a week.
- 2 **Incremental Backup**
  - Only the changes made since last backup (full or incremental) are copied.
  - Takes less storage space.
  - Restoration requires the last full backup and all incremental backups.
  - Example : Daily incremental backup.
- 3 **Differential Backup**
  - Only the changes made since last full backup are copied.
  - Requires more storage than incremental but less than full backup.
  - Restoration requires the last full backup and the latest differential backup.
  - Example : Weekly full backup + daily differential.

## 7. Types of Backup (Diagram)



## 8. Recovery Process

- 1 **Identify the Problem**  
Find the cause of data loss or failure.
- 2 **Choose the Backup**  
Select the appropriate backup (full, incremental or differential).
- 3 **Restore the Data**  
Copy the backup data to the system.
- 4 **Apply Logs (if needed)**  
Use transaction logs to recover the database to a consistent state.
- 5 **Verify the Recovery**  
Check data integrity and consistency.
- 6 **Resume Operations**  
Make the system available for users.

## 9. Importance of Backup and Recovery

- Protects valuable data from loss.
- Ensures continuous business operations.
- Helps in quick recovery after failures or disasters.
- Maintains customer trust and confidence.
- Reduces financial losses.
- Meets legal and regulatory requirements.
- Supports disaster recovery planning.
- Essential for organizations that handle critical and sensitive data.

## 10. Example Scenario

Suppose a company maintains its sales database. Due to a hardware failure, the database becomes unavailable. The company uses a full backup taken on Sunday and incremental backups taken on Monday and Tuesday. By restoring the full backup and applying the incremental backups, the database is recovered up to Tuesday, and business operations resume with minimal data loss.

## 11. Summary

- Backup and recovery is an essential part of DBMS. It protects data from various types of failures.
- Different backup methods (full, incremental, differential) are used based on storage and recovery needs.
- The recovery process restores the database to a consistent state.
- It ensures data availability, business continuity and minimizes downtime.
- Backup and recovery is necessary for reliable and secure database management.



# Archive (Transaction) Logs

## 1. Introduction to Archive (Transaction) Logs

- A transaction log (also called archive log) is a record of all changes made to the database.
- It contains information about every transaction that modifies the data, such as INSERT, UPDATE, DELETE, and DDL operations.
- Before the actual changes are written to the database files, the details are first written to the transaction log.
- The log acts as a history list of all database activities and helps in recovery, auditing and maintaining data consistency.
- In other words, archive (transaction) logs store a chronological record of changes so that the database can be restored to a consistent state in case of failure.

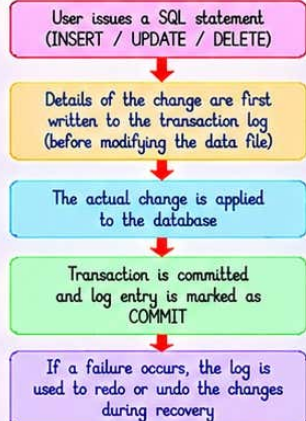
## 2. Why are Transaction Logs Needed?

- Recovery Support**  
Helps to restore the database after crashes or failures.
- Data Consistency**  
Ensures that committed transactions are not lost.
- Audit and Tracking**  
Keeps a history of all changes for monitoring and security.
- Rollback Support**  
Helps to undo incomplete or failed transactions.
- Point-in-Time Recovery**  
Allows recovery of the database to any specific time.
- Compliance**  
Required for legal and regulatory purposes to track data changes.

## 3. Information Stored in Transaction Logs

| Field                    | Description                                        |
|--------------------------|----------------------------------------------------|
| Transaction ID (TID)     | Unique identifier for each transaction.            |
| Operation Type           | Type of operation (INSERT, UPDATE, DELETE, DDL).   |
| Table Name               | Name of the table affected.                        |
| Old Value (Before Image) | Previous value of the data (before modification).  |
| New Value (After Image)  | Updated value of the data (after modification).    |
| Time Stamp               | Time at which the transaction occurred.            |
| Transaction Status       | Shows whether the transaction is committed or not. |
| User/Session ID          | User who performed the operation.                  |

## 4. Working of Transaction Logs



## 5. Types of Transaction Logs

| Type               | Description                                                                               |
|--------------------|-------------------------------------------------------------------------------------------|
| <b>Redo Log</b>    | Used to redo committed transactions that were not written to the database before failure. |
| <b>Undo Log</b>    | Used to undo uncommitted or incomplete transactions.                                      |
| <b>Archive Log</b> | A permanent copy of online redo logs stored for long-term recovery and disaster recovery. |
| <b>Online Log</b>  | Active log files used to record current transactions.                                     |

## 6. Example of a Transaction Log Entry

| TID | Time Stamp           | Operation | Table   | Old Value   | New Value       | Status |
|-----|----------------------|-----------|---------|-------------|-----------------|--------|
| 101 | 03-OCT-2026 10:15:21 | INSERT    | Student | -           | (1, 'Aman')     | COMMIT |
| 102 | 03-OCT-2026 10:16:05 | UPDATE    | Student | 'Aman'      | 'Arman'         | COMMIT |
| 103 | 03-OCT-2026 10:17:10 | DELETE    | Student | (2, 'Riya') | -               | COMMIT |
| 104 | 03-OCT-2026 10:18:00 | UPDATE    | Course  | 'Java'      | 'Advanced Java' | COMMIT |

## 7. Transaction Log and Database Recovery

- Redo Process**: After a system crash, the database uses the log to redo all committed transactions that were not yet written to the database files.
- Undo Process**: It undoes all uncommitted or incomplete transactions to maintain consistency.
- The database scans the transaction logs to find which transactions were committed and which were not.
- Committed transactions are redone and uncommitted transactions are undone.
- Archive logs are especially useful for point-in-time recovery, allowing the database to be restored to any specific moment.

## 8. Archiving of Transaction Logs

- Transaction logs are continuously generated as the database processes transactions.
- To prevent logs from growing too large, older logs are archived.
- Archiving means copying the online redo logs to a separate location (archive storage).
- Archived logs are kept for a long time and are used during media recovery or point-in-time recovery.
- Once a log is archived, it can be reused for new transactions.

## 9. Advantages of Transaction Logs

- Helps in database recovery.
- Maintains data consistency and integrity.
- Provides audit trail of all changes.
- Supports point-in-time recovery.
- Useful for backup and replication.
- Helps in debugging and error analysis.
- Ensures compliance with security and legal requirements.

## 10. Disadvantages of Transaction Logs

- Consumes additional storage space.
- Continuous logging may affect system performance.
- Log files need regular archiving and management.
- If logs are lost or corrupted, recovery may become difficult.
- Large volume of logs in high-transaction systems can be hard to handle.
- Requires proper configuration and maintenance.

## 11. Summary

- Transaction logs (archive logs) record all changes made to the database.
- They store details of each transaction (INSERT, UPDATE, DELETE, etc.).
- Logs are essential for recovery, auditing, and maintaining data consistency.
- There are different types of logs: redo, undo, archive and online logs.
- Archived logs help in long-term and point-in-time recovery.
- Although logs provide many benefits, they also consume storage and require proper management.



# Importance of Backups

## 1. Introduction to Backups

- A **backup** is a copy of the database stored at a separate and secure location.
- It is created to protect data from loss due to various failures, disasters or human errors.
- Backups are an essential part of database management and are used in the **recovery process** to restore the database to a consistent state.
- Without backups, lost or corrupted data may be permanent and cannot be recovered.

## 2. Why Backups are Important ?



A backup ensures that a copy of the database is always available, so that the data can be restored in case of any failure or disaster.

## 3. Situations Where Backups are Needed

- Hardware Failure**: Disk crash, server failure or storage device damage.
- Software Failure**: Corruption due to software bugs or system errors.
- Human Error**: Accidental deletion or incorrect update of data.
- Natural Disasters**: Fire, flood, earthquake, etc.
- Security Threats**: Hacking, virus attack, ransomware, malicious activities.
- Data Corruption**: Due to power failure or unexpected system shutdown.
- Business Continuity**: To ensure uninterrupted operations and quick recovery.

## 4. Importance of Backups (Detailed Points)

### 1. Protects Against Data Loss

- Prevents permanent loss of important and valuable data.
- Helps in restoring data in case of accidental deletion, corruption or hardware failure.



### 2. Ensures Business Continuity

- Helps the organization continue its operations even after a disaster.
- Reduces downtime by quickly restoring the database.



### 3. Recovery from Disasters

- Provides data recovery in case of natural disasters like fire, flood, earthquake, etc.
- Helps in restoring the system to a previous state.



### 4. Protection from Human Errors

- Helps recover data lost due to accidental deletion or incorrect updates.
- Allows rollback to a previous consistent state.



### 5. Security and Compliance

- Helps in recovering from security threats such as hacking, virus attacks or ransomware.
- Helps in meeting legal and regulatory requirements for data retention and recovery.



### 6. Maintains Data Consistency

- Ensures that the recovered database is in a consistent and valid state.
- Helps in avoiding data mismatches and corruption.



### 7. Supports Point-in-Time Recovery

- Allows the database to be restored to a specific time before the failure occurred.
- Useful for undoing incorrect transactions or changes.



### 8. Reduces Financial Losses

- Minimizes the cost of data loss and system downtime.
- Helps in quick recovery, which reduces the financial impact on the organization.



### 9. Helps in Testing and Development

- Backup copies can be used for testing new applications or updates.
- Provides a safe environment without affecting the original data.



### 10. Provides Peace of Mind

- Gives confidence that data is safe and can be recovered when needed.
- Reduces the risk and stress of data loss during unexpected events.



## 5. Examples Showing Importance of Backups

| Scenario                      | Importance of Backup                                                      |
|-------------------------------|---------------------------------------------------------------------------|
| Disk crash                    | Restore the database from backup and continue operations quickly.         |
| Accidental deletion           | Recover the deleted data from backup.                                     |
| Virus attack / Ransomware     | Restore clean data from backup to remove the malicious changes.           |
| Natural disaster (fire/flood) | Recover the database from off-site backup and resume business operations. |
| Incorrect update              | Use backup to rollback to the previous correct state.                     |
| Legal requirement             | Maintain backup for auditing and compliance purposes.                     |
| System upgrade                | Use backup to restore data in case the upgrade fails.                     |

## 6. Summary

- Backups are essential for protecting data from loss, corruption and various types of failures.
- They ensure business continuity, quick recovery, data consistency and compliance with legal requirements.
- Without backups, the data may be permanently lost, which can cause financial and operational problems.
- Therefore, regular and proper backups are very important for every organization.



# Database Recovery and Transaction Control

## 1. Introduction

- A database may fail due to hardware failure, software errors, human mistakes or natural disasters.
- Database recovery is the process of restoring the database to a correct and consistent state after a failure.
- Transaction control ensures that database operations are performed reliably, maintaining data integrity and consistency even in case of failures.
- Together, recovery and transaction control form the core of a reliable DBMS.

## 2. What is a Transaction ?

- A transaction is a sequence of one or more database operations treated as a single logical unit of work.
- It either completes all its operations successfully or none of them (atomicity).
- Transactions are used to maintain data integrity and consistency.

### Example :

Transfer ₹500 from Account A to Account B

- Read balance of A
- Deduct ₹500 from A
- Add ₹500 to B
- Commit the transaction

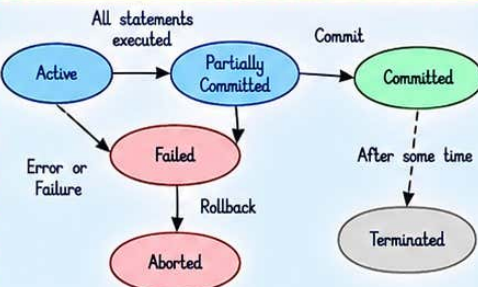
## 3. Need for Transaction Control

- Ensures data consistency when multiple users access the database simultaneously.
- Prevents interference between transactions.
- Maintains isolation so that partial updates are not visible to other users.
- Supports recovery in case of failures.
- Helps in maintaining data integrity and accuracy.
- Avoids problems like lost update, dirty read, non-repeatable read, etc.

## 4. ACID Properties of Transactions

| Property    | Description                                                                              |
|-------------|------------------------------------------------------------------------------------------|
| Atomicity   | All operations of a transaction are completed, or none are performed.                    |
| Consistency | A transaction brings the database from one consistent state to another consistent state. |
| Isolation   | Transactions are executed independently without interference from each other.            |
| Durability  | Once a transaction is committed, its changes are permanent even in case of failure.      |

## 5. Transaction States



## 6. Types of Failures

- Transaction Failure** : Due to logical errors, incorrect input or deadlock.
- System Failure** : Due to power failure or OS crash. (volatile memory lost)
- Media Failure** : Due to disk crash or storage device damage.
- Human Errors** : Accidental deletion or incorrect updates.
- Natural Disasters** : Fire, flood, earthquake, etc.

## 7. Database Recovery

- Database recovery is the process of restoring the database to a correct and consistent state after a failure.
- It uses backup copies, transaction logs and recovery algorithms.
- The goal is to ensure minimal loss of data and to bring the database to the last consistent state.
- Recovery can be automatic or manual depending on the failure type.

## 8. Recovery Techniques

- Log-based Recovery** : Uses transaction logs to undo or redo changes.
- Checkpointing** : Saves the state of the database at regular intervals to speed up recovery.
- Rollback** : Undo the changes of uncommitted transactions.
- Redo** : Reapply the changes of committed transactions not yet written to disk.
- Backup and Restore** : Uses backup copies of the database to recover data.

## 9. Transaction Log

- A transaction log (archive log) is a sequential record of all changes made to the database.
- It stores information such as transaction ID, operation type (INSERT, UPDATE, DELETE), old value and new value.
- Before the actual change is applied to the database, the details are first written to the log (Write-Ahead Logging).
- The log is used for recovery, rollback and auditing purposes.

## 10. Write-Ahead Logging (WAL)

- In WAL, the log record is written to disk before the actual changes are made to the database.
- This ensures that the log always contains a record of every change.
- Even if the system fails, the database can be recovered using the log.



## 11. Recovery Process (Steps)

- Identify the failure type.
- Check the transaction log.
- Undo uncommitted transactions (rollback).
- Redo committed transactions (if required).
- Bring the database to a consistent state.

## 12. Importance of Recovery and Transaction Control

- Ensures data safety and security.
- Maintains consistency and integrity of the database.
- Provides quick recovery after failures.
- Supports continuous availability of the system.
- Reduces the risk of data loss and financial damage.
- Essential for multi-user environments where concurrent transactions occur.
- Builds user trust and reliability in database systems.